



Fakultät Informatik

**Entwicklung eines VHDL-basierten Applikationsbeispiels
zur Ablösung des FM350-2 Moduls durch das TM FAST
Modul inklusive Integration in das TIA Portal**

Bachelorarbeit im Studiengang Informatik

vorgelegt von

Jan-Eric Gedicke

Matrikelnummer 3643446

Erstgutachter: Prof. Dr. Meitner

Betreuer: Conrad, Maximilian

© 2025

Inhaltsverzeichnis

1	Einleitung	1
1.1	Problemstellung und Motivation	1
1.2	Zielsetzung der Arbeit	1
1.3	Methodisches Vorgehen und Struktur der Arbeit	2
2	Grundlagen	3
2.1	Automatisierungspyramide	3
2.2	Überblick über SIMATIC S7	4
2.3	Aufbau eines SIMATIC S7-Systems	5
2.4	Funktionsweise einer Speicherprogrammierbaren Steuerung	6
2.5	Technologiemodule	7
2.6	Programmierung einer PLC mit TIA-Portal	9
2.7	Software	10
2.8	Grundlagen zu Inkrementalgebern	14
2.9	Maybe	16
3	Analyse der Ablösung des FM350-2 durch das TM FAST Modul	17
3.1	Eigenschaften und Aufbau des TM FAST	17
3.2	Funktionsweise des FM350-2	25
3.3	Direkter Vergleich der TM Fast und FM 350-2	37
4	Entwicklung des VHDL-basierten Applikationsbeispiels	39
4.1	Anforderungen an die Neuentwicklung	39
4.2	Modularisierung und Wiederverwendbarkeit	39
4.3	Flexibilität der Kanalzuordnung	40
4.4	Ressourcenoptimierung	41
4.5	Fehler- und Diagnosestrategien	42
4.6	Erweiterbarkeit für zukünftige Anforderungen	45
4.7	Testbarkeit und Simulation	46
4.8	Zyklische und azyklische Kommunikation	47

4.9	Sicherheit bei CPU_STOP	49
4.10	Konzept und Architektur des neuen Applikationsdesigns	50
4.11	Überlaufkompensation (Dynamic Overflow Compensation)	51
4.12	Analyse der Überlaufkompensation	53
4.13	Verteilung von RS485- und DIQ-Kanälen: Flexibilität und Varianten	56
4.14	Umsetzung der Funktionalität in VHDL	57
4.15	Test und Validierung der VHDL-Implementierung	58
4.16	Schnittstelle mit dem TIA Portal	58
4.17	Vergleich mit der bisherigen Lösung	58
5	Entwicklung des VHDL-basierten Applikationsbeispiels	59
5.1	Anforderungen und Architektur	59
5.2	Modularisierung und Flexibilität	59
5.3	Ressourcenoptimierung	60
5.4	Fehler- und Diagnosestrategien	60
5.5	Synchronisation und Zustandsautomaten	60
5.6	Erweiterbarkeit und Testbarkeit	61
5.7	Kommunikationskonzept: Zyklisch vs. azyklisch	61
5.8	Sicherheit bei CPU_STOP	61
5.9	Überlaufkompensation	61
5.10	Flexible Kanalverteilung	62
5.11	Vergleich mit bisherigen Lösungen	62
5.12	Test und Validierung	62
5.13	Fazit	62
6	Fazit und Ausblick	63
6.1	Zusammenfassung der Ergebnisse	63
6.2	Kritische Reflexion und Herausforderungen	63
6.3	Potenzielle Weiterentwicklungen und zukünftige Anwendungen . . .	63
7	Plan	66
8	Fragen	69
9	Wichtig	72
10	TODO	74
11	Tests	76

1 Einleitung

1.1 Problemstellung und Motivation

Die Produktpalette der Firma Siemens im Bereich der Industrieautomatisierung hat eine auslaufende Produktserie vom Typ S7-300, insbesondere das FM350-2 Modul, sind nicht mehr in der aktiven Vermarktung und dadurch nicht mehr bestellbar. Das FM350-2 Modul bietet 8 Kanäle für hochkanalige Dosieranwendungen und ist nur noch als Ersatzteil verfügbar. Ein alternatives Lösungskonzept ist der Einsatz des TM FAST Moduls, eines freiprogrammierbaren Hochgeschwindigkeits-Moduls, welches eine erweiterte Kanalanzahl von 10 bis 12 Kanälen ermöglicht für die neuere Produktserie vom Typ S7-1500. Ziel dieser Bachelorarbeit ist die Entwicklung eines VHDL-basierten Applikationsbeispiels zur Demonstration der Einsatzmöglichkeiten des TM FAST Moduls in hochkanaligen Dosieranwendungen. Gleichzeitig soll die Integration in das TIA Portal erfolgen, um eine reibungslose Nutzung in industriellen Automatisierungsumgebungen zu gewährleisten.

Als dualer Student bei Siemens verfüge ich bereits über Erfahrung in der industriellen Automatisierungstechnik. Diese Arbeit bietet mir die Gelegenheit, mein Wissen in den Bereichen Embedded Systems, Industrieautomation und Software Engineering weiter zu vertiefen. Ich bin Motiviert, dieses praxisnahe Thema in Kooperation mit Siemens zu bearbeiten und einen wertvollen Beitrag zur Weiterentwicklung industrieller Automatisierungslösungen zu leisten.

1.2 Zielsetzung der Arbeit

Ziel dieser Bachelorarbeit ist die Entwicklung eines VHDL-basierten Applikationsbeispiels zur Ablösung des FM350-2 Moduls der alten S7-300-Produktserie durch

das TM FAST Modul der neueren S7-1500-Produktserie. Dabei soll die Kanalanzahl von 8 auf 10 oder 12 erhöht werden, um eine flexiblere Nutzung in hochkanaligen Dosieranwendungen zu ermöglichen. Ein wesentlicher Bestandteil der Arbeit ist zudem die Integration des entwickelten Applikationsbeispiels in das TIA Portal, die Programmiersoftware für die industriellen Steuerungen der Firma Siemens, um eine nahtlose Einbindung in bestehende Automatisierungslösungen zu gewährleisten. Neben der Implementierung des VHDL-Codes werden bewährte Methoden des Software Engineerings angewendet, um die Codequalität und Skalierbarkeit sicherzustellen. Schließlich soll eine umfassende Dokumentation erstellt werden, die eine detaillierte Beschreibung der Nutzung und Inbetriebnahme des Moduls umfasst.

1.3 Methodisches Vorgehen und Struktur der Arbeit

Zur Umsetzung der Ziele werden zunächst die bestehenden Funktionalitäten des FM350-2 Moduls analysiert und an den aktuellen Erfordernissen der Kunden gespiegelt. Dabei werden sowohl die Hardware- als auch die Software-Architektur untersucht und relevante Funktionen für die Dosieranwendung identifiziert. Anschließend erfolgt die Entwicklung des VHDL-basierten Applikationsbeispiels, wobei die ausgewählten Funktionen des FM350-2 Moduls in VHDL umgesetzt und die Kanalanzahl auf 10 oder 12 erweitert wird. Danach wird die Integration in das TIA Portal vorgenommen, indem das TM FAST Modul an das Automatisierungssystem angebunden und geeignete Schnittstellen sowie Datenstrukturen entwickelt werden. Qualitätssicherungsmaßnahmen spielen ebenfalls eine zentrale Rolle. Hierbei werden bewährte Entwicklungswerkzeuge wie Intel Quartus Prime und das TIA Portal genutzt, Teststrategien zur Funktionssicherung implementiert und eine modulare Softwarearchitektur entworfen. Abschließend wird die Dokumentation erstellt, die eine umfassende Beschreibung der Implementierung und Nutzung des Moduls sowie Anwendungsbeispiele und Testberichte enthält. Die Ergebnisse werden in einer Abschlusspräsentation vorgestellt.

2 Grundlagen

Um die technischen Aspekte und die Funktionsweise der in dieser Arbeit behandelten Systeme zu verstehen, werden im Folgenden die zugrundeliegenden Konzepte und Technologien erläutert. Relevant sind hierbei die Automatisierungspyramide, SIMATIC S7, Speicherprogrammierbare Steuerungen, Quartus und Questa sowie TIA Portal und ?Echtzeitsysteme?.

2.1 Automatisierungspyramide

Die Automatisierungspyramide stellt ein Konzept dar, welches die Hierarchie der Automatisierungstechnik von der Prozess- bis hin zur Unternehmensleitebene beschreibt.

Die Automatisierungspyramide besteht aus insgesamt fünf hierarchischen Ebenen. Diese Ebenen lassen sich in die Systemgruppen IT als Unternehmensleitebene und OT als Fertigungsebene unterteilen [4, S. 4 f.].

Ebenen der OT

- **Ebene 0 – Prozessebene:** Darstellung eines verfahrenstechnischen oder mechanischen Prozesses als unregelmäßiges System.
- **Ebene 1 – Feldebene:** Interaktion mit der Umgebung, Sensorik und Aktorik eines Systems.
- **Ebene 2 – Steuerungsebene:** Koordination von Anlagen und Geräten durch Steuereinheiten wie SPS (PLCs).
- **Ebene 3 – Prozessleitebene:** Überwachung und Ressourcenoptimierung der Gesamtproduktion.

Ebenen der IT

- **Ebene 4 – Betriebsleitebene:** Sammlung und Auswertung von Daten der Prozessleitebene mehrerer Gesamtproduktionen. Koordination zwischen mehreren Abteilungen.
- **Ebene 5 – Unternehmensebene:** Festlegung langfristiger Ziele des Unternehmens mithilfe der erhobenen Daten aus der Betriebsleitebene [4].

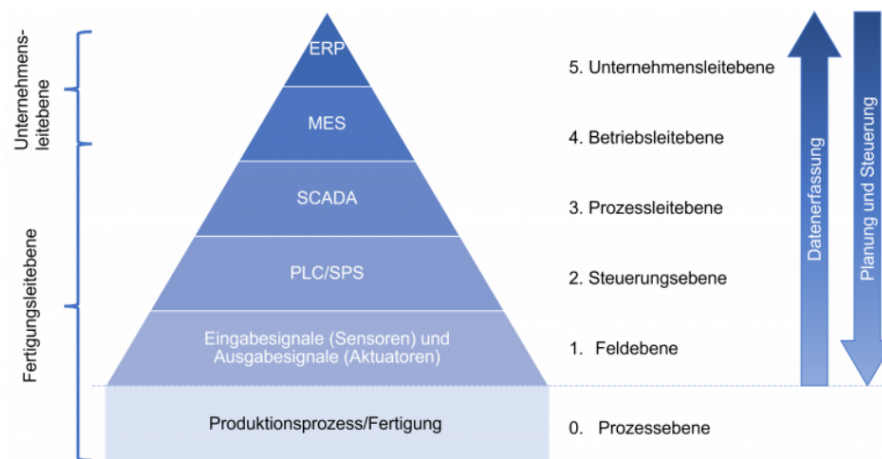


Abbildung 2.1: Automatisierungspyramide, [4]

Das in dieser Arbeit behandelte System findet somit in den Ebenen 1 bis 3 statt. In der Feldebene befindet sich die Sensorik und Aktorik, deren Signale durch eine Steuerungssoftware auf der TM FAST verarbeitet werden sollen. In der Steuerungsebene findet sowohl die Prozessdatenverarbeitung als auch die Kommunikation zwischen dem TM FAST und der PLC statt. Die Visualisierung befindet sich in der Prozessleitebene.

2.2 Überblick über SIMATIC S7

Speicherprogrammierbare Steuerungen – im Englischen „Programmable Logic Controllers“ (PLCs) – wie die SIMATIC S7-Serie von Siemens steuern und regeln unter anderem Maschinen, Antriebe, Ventile, Pumpen und Sensoren. Diese Steuerungen sind frei programmierbar und dadurch universell einsetzbar [[6], S. 146]. Im Folgenden wird ein Überblick über den Aufbau, die Funktionsweise und die Programmierung eines SIMATIC S7-Systems gegeben.

2.3 Aufbau eines SIMATIC S7-Systems

Ein SIMATIC S7-System ist modular aufgebaut und besteht aus mehreren Komponenten. Im Folgenden werden die Hauptkomponenten und ihre Funktionen erläutert.

Controller (CPU)

Die CPU der Steuerung wird auch als Controller bezeichnet und bildet die zentrale Verarbeitungseinheit des Systems. Die Hauptaufgabe der CPU ist es, das Anwenderprogramm auszuführen, das vom Nutzer entwickelt wurde, um spezifische Steuerungs- und Regelungsaufgaben zu übernehmen [[6], S. 146]. Die CPU verarbeitet Signale von Ein- und Ausgabemodulen, führt Berechnungen durch und gibt Steuerbefehle an angeschlossene Geräte, wie Sensoren und Aktoren, aus.

Controller der SIMATIC S7 Serie verfügen über integrierte Kommunikationsschnittstellen, wie Ethernet/PROFINET oder PROFIBUS, die den Datenaustausch mit anderen Steuerungen, Bediengeräten oder übergeordneten Systemen (z. B. SCADA oder MES) ermöglichen [12, S. 14].

Weitere Baugruppen

Um eine Verbindung zur Maschine oder Anlage herzustellen, werden Signalbaugruppen (Eingabe und Ausgabebaugruppen) eingesetzt, die digitale oder analoge Prozesssignale (z. B. Spannungen, Ströme) erfassen bzw. ausgeben. Sie stellen selbst keine komplexe Vorverarbeitung bereit, sondern bilden primär die Schnittstelle zwischen Feld und Steuerung.

Technologiebaugruppen (Technologiemodule) gehen darüber hinaus: Es handelt sich um Erweiterungsbaugruppen mit eigener hardwarenaher Logik (z. B. FPGA oder Mikrocontroller), die abhängig von ihrer Parametrierung im TIA Portal definierte Funktionen autark und ereignisgesteuert ausführen können, ohne dass für jeden Schritt ein zyklischer Eingriff der CPU (OB1) erforderlich ist. Dadurch entfallen die durch die CPU Zykluszeit bedingten Latenzen und Jitter und zeitkritische Operationen (z. B. High Speed Zählen, präzise Zeitstempel, Pulsmessungen und Frequenzmessungen, Positionserfassung, schnelle Reaktionsausgänge) können mit deutlich geringerer Reaktionszeit und höherer Deterministik verarbeitet werden. Die CPU wird entlastet und erhält bereits vorveredelte Daten, was Ressourcen

spart und die Gesamtperformance erhöht. Genau dieser Vorteil (schnelle deterministische Reaktion ohne Einfluss der CPU Zykluszeit) motiviert den Einsatz von Technologiemodulen.

Kommunikationsbaugruppen erweitern ergänzend die Protokollfähigkeit oder Schnittstellenfähigkeit einer Steuerung (z. B. zusätzliche Feldbusse oder Industrial Ethernet Varianten) und kapseln entsprechende Kommunikationsstacks von der CPU [[6], S. 146].

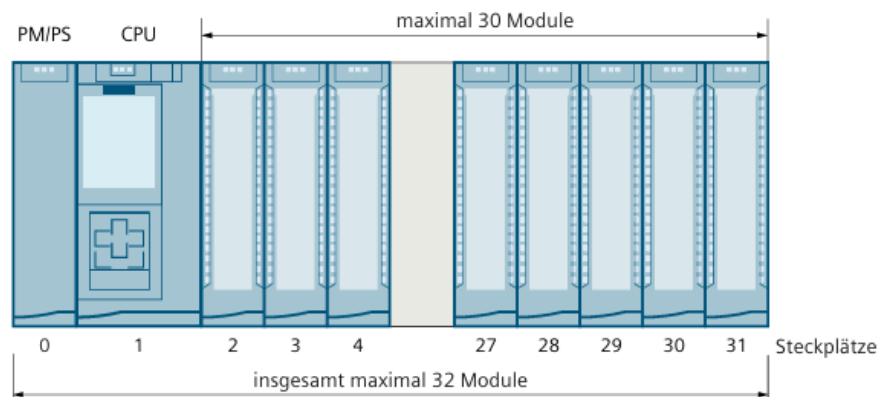


Abbildung 2.2: Aufbau SIMATIC S7-System, [9]

2.4 Funktionsweise einer Speicherprogrammierbaren Steuerung

Grundsätzlich arbeitet eine Speicherprogrammierbare Steuerung nach dem Eingabe-Verarbeitung-Ausgabe-Prinzip (EVA). Der damit verbundene Prozess erfolgt zyklisch und lässt sich wie folgt darstellen:

Über Eingangsbaugruppen werden zunächst die Zustände von angeschlossener Sensorik erfasst. Diese Eingangssignale werden anschließend in einem Prozessabbild gespeichert, welches eine Momentaufnahme der aktuellen Signalzustände darstellt. Änderungen der Signale, die während eines laufenden Zyklus auftreten, werden somit ignoriert [[6], S. 147], [13, S. 39].

Das Prozessabbild dient nun als Datengrundlage für die Verarbeitung durch das Anwenderprogramm. Das Anwenderprogramm ist modular aufgebaut und arbeitet mit sogenannten Bausteinen. Das Hauptprogramm wird durch den Organisa-

tionsbaustein OB1 repräsentiert, welcher aus konkreten Anweisungen aber auch Funktions- oder Unterprogrammaufrufen besteht. Die Abarbeitung des OB1 entspricht einem Verarbeitungszyklus (Scan). Dessen Dauer liegt typischerweise im ein- bis zweistelligen Millisekundenbereich, kann aber – abhängig von Programmlogik, umfangreicher Kommunikation, vielen Zugriffen auf Peripherie oder aktivierten Diagnose-/Tracing-Funktionen – auch deutlich ansteigen und in den hohen zweistelligen bis dreistelligen Millisekundenbereich reichen. Überschreitet die Zykluszeit das konfigurierte Watchdog-Limit, wird ein Zeitfehler ausgelöst (Fehler-OB, z. B. OB80) [13, S. 39].

Wichtig ist, dass OB1 nicht strikt ununterbrochen läuft: Er kann durch höher priorisierte Organisationsbausteine (Interrupt-/Alarm-OBs) präemptiv unterbrochen werden. Dazu zählen u. a. zeitgesteuerte Interrupt-OBs (z. B. OB35 für einen zyklischen Zeit-Interrupt), Hardware- bzw. Prozessalarm-OBs (OB40ff) oder bestimmte Kommunikations- und Diagnose-OBs. Die Ausführung dieser höher priorisierten OBs verschiebt das Fertigstellen des aktuellen OB1-Scans nach hinten und verlängert damit die effektive Zykluszeit von OB1; die Bearbeitung der Interrupt-OBs selbst erfolgt jedoch deterministisch gemäß ihrer Priorität.

Nach der (ggf. unterbrochenen und dann fortgesetzten) Verarbeitung des Anwenderprogramms erstellt die SPS ein neues Prozessabbild, das die Ergebnisse der Programmverarbeitung enthält. Dieses Abbild wird schließlich an die Ausgangsbaugruppen übertragen, woraufhin die angeschlossenen Aktoren angesteuert werden. Mit diesem Schritt ist der Zyklus abgeschlossen, und die SPS beginnt unmittelbar mit der Erfassung der neuen Eingangs-Signalzustände für den nächsten Zyklus [[6], S. 147], [13, S. 39].

2.5 Technologiemodule

Die SIMATIC S7-1500 Steuerung ist ein modulares System, das in ihrer Funktionalität frei skalierbar ist. Den Kern der Steuerung bildet die SIMATIC S7-1500 CPU. In ihr wird das Anwenderprogramm ausgeführt und die Verbindung zu anderen Automatisierungskomponenten aufgebaut. Neben der CPU kann das System mit Interface-, Signal-, Kommunikations- und Technologiebaugruppen erweitert werden. Das sogenannte Peripheriesystem ET 200MP ermöglicht den Einsatz der S7-1500 IO-Module entweder, wie beschrieben, direkt an der SIMATIC S7-1500

Steuerung oder an einem entsprechend kompatiblen PROFINET-/PROFIBUS-Teilnehmer, wie z.B. dem Interfacemodul IM 155 PN, das die Daten zwischen Steuerung und Peripheriemodulen austauscht. Abb. 1 zeigt zum einen auf der linken Seite den zentralen Aufbau mit CPU und vier Signalbaugruppen und zum anderen auf der rechten Seite den vergleichbaren dezentralen Aufbau mit Interfacemodul. Die Kommunikation erfolgt über PROFINET. Die Bezeichnung Signalbaugruppe fasst die Digital- und Analogbaugruppen zusammen, die der Steuerung zusätzliche Ein- und Ausgänge zur Verfügung stellen.[7, 8]



Abbildung 2.3: SIMATIC ET 200MP im zentralen und dezentralen Aufbau [9]

Technologiebaugruppen bieten hardwarenahe Signalvorverarbeitung für schnelles Zählen und Messen sowie die genaue Positionserfassung für Inkremental- und SSI-Absolutwertgeber. Bei bisherigen Modulen erfolgt die Parametrierung und Programmierung durch Technologieobjekte oder durch Ansteuerung der IOs im Anwendungsprogramm im TIA Portal. Das aktuelle Portfolio umfasst u.a. das TM NPU, TM Count 2x24 V, TM Posinput 2, TM Timer DIDQ 16x24 V und das TM PTO 4. Ersteres arbeitet mit einem trainierten neuronalen System und es können verschiedene Sensoren über Gigabit-Ethernet oder USB 3.1 angeschlossen werden. Das TM Count 2x24 V ist eine vielseitig einsetzbare Zählerbaugruppe für Zähl- und Messaufgaben sowie die Positionserfassung mit Inkrementalgebern. Ein weiteres Zähl- und Positionserfassungsmodul ist das TM Posinput 2. Dieses bietet zwei Zählkanäle für Differenz-Eingangssignale nach RS422 mit bis zu 1 MHz und ermöglicht die Positionserfassung mit SSI-Absolutwertgebern. Das TM Timer DIDQ 16x24 V wird für Aufgaben eingesetzt, die einen besonderen zeitlichen Anspruch haben, wie z.B. die exakte Einhaltung von Reaktionszeiten. Das PTO

4 Modul wird zur Ansteuerung von Schrittmotor-Antrieben und Servo-Antrieben mit PTI-Schnittstelle verwendet.[9]

Das TM FAST ist bereits seit einiger Zeit im Portfolio und wird kontinuierlich in kurzen Iterationen erweitert. Dabei werden neue, aus konkreten Kunden-Use-Cases abgeleitete Funktionen (z.B. zusätzliche Protokoll-/Zähl-/Trigger- und Preprocessing-Bausteine) implementiert und als wiederverwendbare Funktionsblöcke bereitgestellt. Auf diese Weise können bestehende Anforderungen schneller abgebildet und künftige Anwendungsfälle flexibel ergänzt werden. Im nachfolgenden Abschnitt wird das TM FAST genauer vorgestellt.

2.6 Programmierung einer PLC mit TIA-Portal

Das Totally Integrated Automation Portal (TIA Portal) ist ein Framework für die Projektierung, Programmierung und Inbetriebnahme von SIMATIC S7-Steuerungen und deckt unter anderem Hardware-Konfiguration, Softwareentwicklung und Fehlerdiagnose ab [14, S. 11]. Die Programmierung folgt dabei der internationalen Norm IEC 61131-3, die den Standard für die Softwareentwicklung in speicherprogrammierbaren Steuerungen (SPS) definiert. Diese Norm legt die Struktur und die Programmiersprachen fest, die in Automatisierungsprojekten eingesetzt werden können, um eine einheitliche und herstellerübergreifende Programmierbarkeit zu gewährleisten.

Nach IEC 61131-3 werden fünf standardisierte Programmiersprachen für SPS definiert: KOP (Kontaktplan): Eine grafische Sprache, die sich an elektrischen Schaltplänen orientiert. FUP (Funktionsplan): Eine grafische Sprache zur Darstellung von Logik und Funktionen. AWL (Anweisungsliste): Eine textbasierte Sprache, die ähnlich wie Assembler arbeitet. SCL (Structured Control Language): Eine textbasierte, hochsprachliche Sprache ähnlich zu Pascal. AS (Ablaufsprache): Eine grafische Sprache zur Darstellung von Ablaufsteuerungen [[6], S. 153].

Für die Programmierung im Rahmen der Bachelorarbeit sind insbesondere die Sprachen FUP und SCL relevant.

Funktionsplan (FUP)

FUP ist eine grafische Programmiersprache. Sie basiert auf der Darstellung von Funktionsbausteinen, die durch Verbindungen miteinander verknüpft werden.

Ein typisches Beispiel in FUP ist die Implementierung von UND-, ODER- oder NICHT-Verknüpfungen, bei denen Ein- und Ausgangssignale miteinander verbunden werden. Die Funktionsbausteine enthalten logische Funktionen, mathematische Operationen oder auch spezifische Steuerungsaufgaben, die als grafische Symbole dargestellt sind [15, S. 91 f.].

Structured Control Language (SCL)

SCL ist eine textbasierte Programmiersprache, die der Hochsprache Pascal ähnelt. Sie ist besonders für komplexere Algorithmen und mathematische Berechnungen geeignet. SCL wird in der IEC 61131-3 als Strukturierter Text bezeichnet und unterstützt eine strukturierte Programmierung mit Schleifen, Verzweigungen und Funktionsaufrufen [[6], S. 153].

2.7 Software

Bei der Erstellung einer Anwendung für das TM FAST werden verschiedene Softwareprogramme benötigt. Im TIA Portal wird die Hardwarekonfiguration und das Anwenderprogramm, welches die CPU implementiert und den Part im TM FAST aktiviert, erstellt. Zudem kann die Firmware über das Engineering-Framework auf das TM FAST geladen werden. Für die Programmierung des Intel® FPGA Cyclone 10 LP wird Intel® Quartus® Prime Lite verwendet. Um die Funktionalität des Hardwareentwurfs vorab zu testen, wird die zugehörige Questa-Intel® FPGA Starter Edition genutzt.

2.7.1 Totally Integrated Automation Portal

Das Totally Integrated Automation Portal (TIA Portal) ist ein Engineering-Framework, das von Siemens für die Anwendung in verschiedensten digitalen Automatisierungsaufgaben entwickelt wurde. Im TIA Portal kann der vollständige Entwicklungsprozess von der Planung, über das integrierte Engineering bis hin zum transparenten Betrieb eines Automatisierungssystems durchgeführt werden. Die Entwicklungsumgebung umfasst unterschiedliche Softwarepakete, wie z.B. SIMATIC STEP7 für die Programmable Logic Controller (PLC)-Programmierung, SIMATIC WinCC für die Visualisierung von Maschinenprozessen auf Human Machine Interfaces (HMIs), SINAMICS Startdrive zur Parametrierung von Antrieben oder

SIMATIC Energy Suite für die Erfassung von Energiedaten und Überwachung des Energieverbrauchs in der Anlage. In dieser Arbeit wird TIA Portal V20 verwendet [[7], [8]]. Damit die CPU eine Verbindung zu dem Technologiemodul aufbauen kann, ist eine Hardwarekonfiguration im TIA Portal notwendig. Hierfür werden die benötigten Komponenten anhand ihrer Artikelnummer ausgewählt und miteinander verbunden. Abb. 6 zeigt die Hardwarekonfiguration im TIA Portal mit einem Powermodul (PM) zur Spannungsversorgung (Slot 0), einer CPU 1516-3 PN/DP (Slot 1) und einem TM FAST (Slot 2). Da die drei Komponenten auf einer gemeinsamen Schiene angebracht sind, können sie über den Rückwandbus kommunizieren. Es ist keine zusätzliche Verbindung über PROFINET oder PROFIBUS erforderlich.



Abbildung 2.4: Hardwarekonfiguration im TIA Portal mit TM FAST

Das TM FAST besitzt nach der Hardwarekonfiguration im TIA Portal einen eigenen Eintrag. Hier können unter Online & Diagnostics allgemeine Informationen über das Modul ausgelesen, der Diagnose-Status abgefragt und eine neue Firmware auf das FPGA geladen werden. Unter dem Menüpunkt Commissioning kann der Status und die aktuelle User-Logik Version abgefragt werden. Zudem gibt es die Möglichkeit, verschiedene Kommandos an das FPGA zu schicken, wie z.B. das Laden der TM FAST-Anwendung aus dem Flash-Speicher oder die Aktivierung der Debug-Schnittstelle. Die Register der azyklischen Kommunikation können unter dem Punkt TM FAST user data records manuell beschrieben oder ausgelesen werden. Die Funktionsmöglichkeiten zur Steuerung der Anwendung, Lesen und Schreiben von Daten sowie Laden der Anwendung können entweder im Menü des TIA Portals oder als Funktionsbausteine (FB) der TIA-Anweisungsbibliothek LTFAST im Programm ausgeführt werden. Die TIA-Library enthält die Anwei-

sungen LTMFAST ControlREC, LTMFAST UserReadREC, LTMFAST UserWriteREC und LTMFAST AppDownload [6, 17].

2.7.2 Intel® Quartus® Prime

Die Intel® Quartus® Prime Software ist eine Designumgebung für Intel® FPGAs. Es beinhaltet alle Schritte der Entwicklung von Designerfassung, Synthese bis hin zur Optimierung, Verifizierung und Simulation. Von Intel® werden drei verschiedene Versionen angeboten. Neben der kostenlosen Lite Edition gibt es noch die Standard und Pro Edition. Die Versionen unterscheiden sich teilweise im Funktionsumfang sowie in der Unterstützung der FPGA Board Familien. Das FPGA Cyclone 10 LP, das im TM FAST verbaut ist, kann nur in den Editionen Standard und Lite verwendet werden. Für die Umsetzung der Anwendungen in dieser Arbeit wird die Quartus® Prime Version 22.1std.0 Lite Edition verwendet [18]. Das Quartus Projekt, das zur Programmerstellung für das TM FAST dient, kann in verschiedene Revisionen unterteilt werden. Diese beinhalten unabhängig voneinander eine eigene VHDL-Architektur sowie eine Konfigurationsdatei. Die Architektur bestimmt das Verhalten des Programms. In der Konfiguration können Parameter wie z.B. die Frequenz der User-Clock oder der Schnittstellentyp der acht Channel individuell eingestellt werden. Über einen Multiplexer wird anschließend die ausgewählte Architektur der Top-Entity TFL FAST USER e zugeordnet. Die Aufteilung in verschiedene Revisionen ermöglicht es, mehrere Anwendungen zu implementieren, ohne das Programmgerüst aus System- und User-Logik jedes Mal erneut aufzusetzen.

In der Quartus® Prime Software werden mehrere Schritte durchgeführt, um vom geschriebenen VHDL-Code zum fertigen Bitstream, der die Logik der Anwendung für das Technologiemodul beinhaltet, zu gelangen. Der Kompilationsvorgang beginnt mit der Analyse und Synthese. Es wird die logische Vollständigkeit und Konsistenz des Projektes sowie Syntaxfehler geprüft. Anschließend wird die Logik der Dateien der Design Entity synthetisiert und ein Technologie-Mapping durchgeführt. Das bedeutet, es werden Flipflops, Latches und Zustandsautomaten aus dem VHDL-Code abgeleitet und optimiert, sodass der Ressourcenverbrauch minimiert werden kann. In der Analyse und Synthese werden Algorithmen angewandt, um die Anzahl der Gatter zu reduzieren und redundante Logik zu entfernen. Dadurch wird gewährleistet, dass die Architektur des Geräts so effizient wie möglich

genutzt werden kann. Nach der Analyse und Synthese folgt der Fitter. Hier werden die logischen und zeitlichen Anforderungen des Projekts mit den verfügbaren Ressourcen des Zielgeräts abgeglichen. Jeder Logikfunktion wird der beste Logikzellenplatz hinsichtlich des Routings und Timings zugeordnet und geeignete Verbindungspfade und Pinbelegungen ausgewählt [19].

Ein FPGA besteht aus einer großen Anzahl von programmierbaren Logik-Blöcken, die jeweils eine kleine Menge an digitaler Logik und programmierbaren Routing implementieren können. Routing ermöglicht die Verbindung der Ein- und Ausgänge der Logikblöcke zu einer größeren Schaltung. Die Verzögerung einer im FPGA implementierten Schaltung ist hauptsächlich auf die Verzögerung beim Routing zurückzuführen. Zudem wird der größte Anteil der Fläche eines FPGAs dem programmierbaren Routing zugeschrieben [20]. Nach dem Fitter kommt das Assembler-Modul zum Einsatz. Dieses fügt die Post-Fit-Logikpartitionen zusammen. Es entstehen Dateien, die zur Programmierung oder Konfiguration eines FPGAs verwendet werden können. Der Assembler konvertiert automatisch die Baustein-, Logikzellen- und Pin-Zuweisungen des Fitters in ein sogenanntes Programmier-Image in Form eines Programmer Object Files (.pof) oder SRAM Object Files (.sof). Für den Prozess, die Logik auf das TM FAST zu laden, wird ein Raw Binary File (.rbf) benötigt. Dies ist ein Bitstream, der vom Assembler aus den Speicherabbilddateien (Memory Image Files .mif) generiert wird [19]. Die weiteren Schritte des Verfahrens, die in VHDL hinterlegte Anwendungslogik auf das TM FAST zu importieren, wird in Abschnitt 2.3.4 nochmal ausführlicher betrachtet.

In der Quartus® Prime Software gibt es mit dem Tool Signal Tap Logic Analyzer eine Möglichkeit den Hardwareentwurf zu debuggen. Die Software erfasst und zeigt das Echtzeit-Signalverhalten in einem Intel® FPGA Design an. Es kann das Verhalten interner Signale während des Normalbetriebs untersucht werden, ohne zusätzliche Peripherie anschließen zu müssen. Durch Bedingungen können der Start und das Ende der Datenerfassung bestimmt werden. Zudem gibt es die Option eine Triggerbedingung festzulegen. Die aufgenommenen Daten können anschließend abgespeichert und gefiltert werden [21]. Um das Softwaretool nutzen zu können sind die Hardware-Komponenten Intel® FPGA Download Kabel (USB-Blaster) und der TM FAST Debug Connector notwendig. Diese bilden das Debug-Interface zwischen PC und TM FAST. Der Connector wird auf dem Modul angebracht und stellt somit eine Verbindung zur JTAG-Schnittstelle her [10].

WICHTIG: Einfügen eines Bildes wo die TMFAST Struktur zu sehen ist

2.7.3 Questa-Intel® FPGA

Es gibt zwei Software-Editionen zur Simulation von Hardware-Designs. Die kostenpflichtige Questa-Intel® FPGA Software und die Questa-Intel® FPGA Starter Edition. Beide sind eine Version der Siemens EDA Questa Core Software für Intel® FPGAs [22]. Questa bietet eine Simulations-, Debug- und Verifikationsplattform zur Validierung von FPGA-Entwürfen [23]. Die Simulation kann entweder über die Quartus® Prime Software oder direkt in der Questa-Softwareplattform mit einem Simulationsskript gestartet werden. Um eine VHDL-Architektur testen zu können, wird eine Testumgebung, auch Testbench genannt, benötigt. Es werden die Eingangssignale (Stimuli) nachgebildet, um die infolgedessen generierten Ausgangssignale überprüfen zu können. Die Stimuli werden in verschiedenen Prozessen generiert und mit der Instanz des zu testenden Modells verbunden [13]. Ein Prozess kann beispielsweise für die Nachbildung des Clock-Signals zuständig sein. Dies erfolgt durch eine Dauerschleife mit einer Wartezeit, nach deren Ablauf der Signalwert getoggelt wird. Dadurch ergibt sich eine Clock mit der Periode der doppelten Wartezeit. Anschließend kann die Simulation in einem Zeitdiagramm analysiert werden. Abb. 7 zeigt den zeitlichen Verlauf einer Testbench, die den Funktionsablauf der Signum-Funktion überprüft. Eine Signum-Funktion gibt bei einer positiven Zahl den Wert “+1“, bei null den Wert “0“ und bei einer negativen Zahl den Wert “-1“ aus [24]. In der Testbench werden drei signed Werte im Hexadezimal-Format abwechselnd als Input verwendet. Bei einer steigenden Flanke der Clock liegt nach einem Wechsel des Eingangssignals das Ergebnis am Ausgang an.

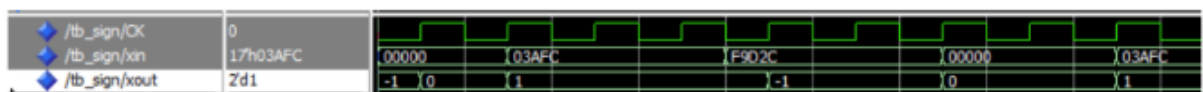


Abbildung 2.5: Zeitdiagramm zur Simulation der Signum-Funktion in einer Testbench in der Questa-Intel® FPGA Starter Edition

2.8 Grundlagen zu Inkrementalgebern

Inkrementalgeber erzeugen aus einer mechanischen Weg- oder Drehbewegung zwei zueinander um 90° phasenverschobene, rechteckförmige Ausgangssignale (A/B;

Quadratur). Bei konstanter Geschwindigkeit entstehen gleichförmige Pulszüge; die Pulsfrequenz ist proportional zur Geschwindigkeit und kann über einen geeigneten Skalenfaktor in Geschwindigkeitswerte umgerechnet werden [13].

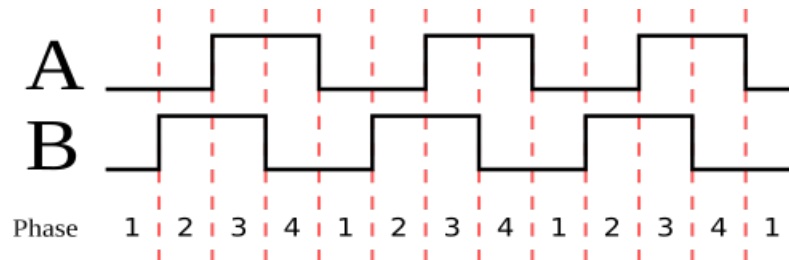


Abbildung 2.6: Zwei Rechteckwellen in Quadratur [12]

Rotative Geber realisieren dies typischerweise mit einer Lichtquelle, einer codierten Scheibe auf der Welle und Photodetektoren. Die Bewegungsrichtung wird aus der Phasenlage der Quadratur-Signale bestimmt: In einer Drehrichtung führt A vor B, in der anderen führt B vor A. Für eine höhere Auflösung kann nicht nur pro Puls, sondern auf jeder Flanke gezählt werden (1x-, 2x- oder 4x-Auswertung); dadurch lässt sich die Zählauflösung auf das bis zu Vierfache erhöhen [12, 13]. Aufgrund dieser Eigenschaften werden Inkrementalgeber häufig in Anwendungen eingesetzt, die eine präzise Messung und Regelung von Position und Geschwindigkeit erfordern.

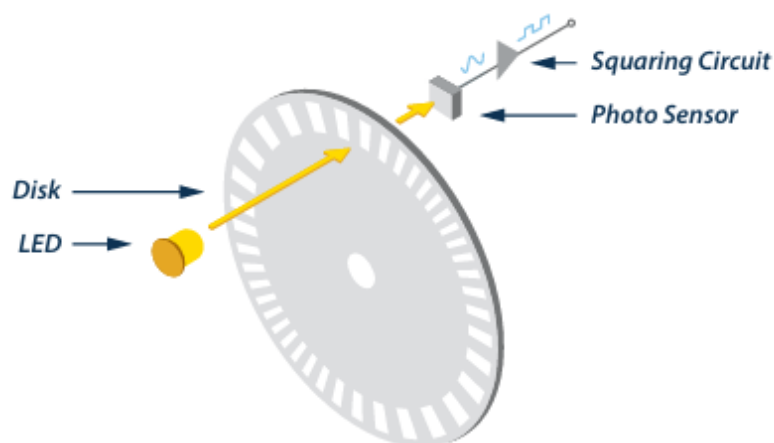


Abbildung 2.7: Funktionsweise eines Inkrementalgebers [12]

2.9 Maybe

-Echtzeitsysteme? Übergang zu was ich mache (ist das wichtig? wahrscheinlich nicht) -24V und TTL Inkremental-Geber erklären?

3 Analyse der Ablösung des FM350-2 durch das TM FAST Modul

In diesem Abschnitt werden die technischen Grundlagen und Eigenschaften der beiden betrachteten Module, FM350-2 und TM FAST, vorgestellt. Es werden deren Aufbau, Funktionsweise, Schnittstellen und typische Einsatzgebiete erläutert. Ziel ist es, die relevanten Technologien und Unterschiede beider Systeme darzustellen, um die Basis für die spätere Analyse der Ablösung des FM350-2 durch das TM FAST Modul zu schaffen.

3.1 Eigenschaften und Aufbau des TM FAST

Das TM FAST bietet die Möglichkeit, eigene Anwendungen in das SIMATIC S7-1500 Technologiemodul einzubetten. Hardwareseitig besteht es aus einem Backend-Baustein, der die Siemens TM FAST-Firmware bereitstellt, und einem Frontend-Baustein (Intel® Cyclone 10 LP FPGA), der die sogenannte TM FAST-Anwendung hostet. Diese Applikation bestimmt die Hardwarefunktionalität des Technologiemoduls und kann durch applikationsspezifisches Programmieren des FPGAs selbst definiert werden. Hierzu wird die externe Software Intel® Quartus® Prime verwendet. Näheres zu den Programmen der Softwareentwicklung und -simulation folgt in Abschnitt 2.3. [Guillaume: Specification for TM Fast, 10]

Eine vergleichbare Baugruppe ist der High Speed Boolean Processor FM352-5 aus dem Automatisierungssystem S7-300. Dieser wird mit einer Zykluszeit von 1 μ s für Anwendungen verwendet, die eine schnelle Binärsteuerung sowie Schaltvorgänge benötigen. Die Funktionsbaugruppe wird in STEP 7 mit Kontaktplan (KOP) oder Funktionsplan (FUP) mit einem zur Verfügung stehenden Anweisungssatz programmiert [11]. Durch die Programmierung des FPGAs des TM FAST in der

Intel® Quartus® Prime Software ergibt sich eine höhere Flexibilität in der Anwendungsprogrammierung und ermöglicht spezifische und benutzerdefinierte Lösungen. Mit einer Zykluszeit von 20 ns und die somit geringere Verarbeitungs- und Reaktionszeit eröffnen sich neue Einsatzmöglichkeiten. Es können beispielsweise beliebige Pulsmuster an den Ausgängen erzeugt werden, die zeitlich sehr fein aufgelöst werden können (unter 1 μ s) und in Abhängigkeit.

Im Gegensatz zur einfachen Verarbeitung digitaler Signale kann mit dem TM FAST auch die Auswertung anderer Signale, wie z. B. eines Zählstands, erfolgen. Nachfolgend sind weitere Beispiele aufgelistet. [[2], [3]]

- Positionsabhängige Nocken
- Sehr schnelle Reaktionen auf digitale Ereignisse
- Zählen von Signalen bis zu 5 MHz
- Erfassungslogiken für Inkremental- und Absolutwertgeber mit Ableitung einer unmittelbaren Reaktion
- Präzise Ansteuerung sowie flexible Steuerung von Lasern in verschiedenen Anwendungsfällen
- Pixelgenaue Punktmuster auf mehreren parallelen Spuren
- Exakte Dosieraufgaben
- Durchführung von Berechnungen und Vorverarbeitungen innerhalb des Moduls, z. B. zur Fehlerfrüherkennung

Aufgrund der freien Programmierbarkeit können einzelne Funktionen variabel angepasst und kombiniert werden. Zusammenfassend eignet sich das TM FAST für alle Aufgaben, bei denen die speicherprogrammierbare Steuerung (SPS) mit Peripheriebaugruppe entweder nicht schnell genug ist oder die nicht durch ein vorhandenes Technologiemodul umgesetzt werden können und daher einer speziellen Einzellösung bedürfen.

Der **Aufbau** des TM FAST aus Backend-Gerät, Frontend-Gerät, Prozessschnittstelle sowie die Anbindung an die CPU und Peripherie wird in Abb. 2 veranschaulicht.

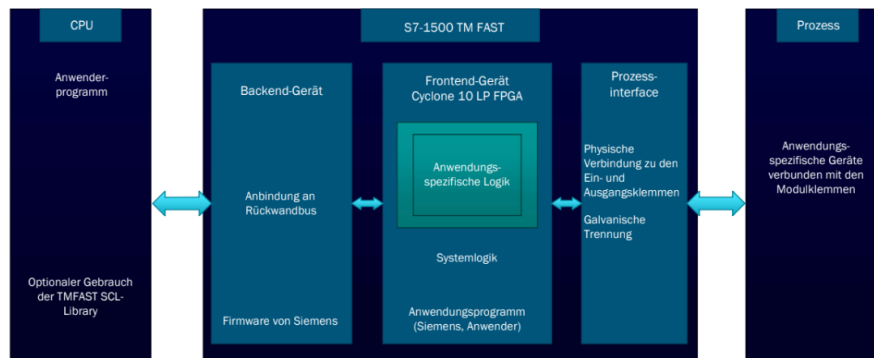


Abbildung 3.1: Abbildung 6: Aufbau des TM FAST als Blockdiagramm [Guillaume: Specification for TM Fast]

Das **Backend-Gerät** realisiert die Verbindung des TM FAST zu anderen SIMATIC

Komponenten über den Rückwandbus der zugehörigen SIMATIC S7-1500 Automatisierungseinheit. Die entsprechende Firmware wird von Siemens bereitgestellt und weiterentwickelt. Diese regelt u.a. die Kommunikation mit der S7-1500 Steuerung, den zyklischen und azyklischen Datentransfer sowie den Diagnosestatus. Genaues zur Kommunikation folgt im späteren Verlauf des Abschnittes. Des Weiteren wird im Backend-Gerät die TM FAST-Applikation aus dem Flash geladen, über die SPI-Schnittstelle auf das FPGA übertragen, validiert und nach Bestätigung aktiviert. Zudem wird das Frontend-Gerät beobachtet, um Fehlverhalten zu erkennen. Sind die Parameter des Modules gesetzt, wird durch die Firmware die Systemlogik entsprechend angepasst. Das Backend-Gerät liest die Temperatur des Moduls über den Temperatursensor aus und kontrolliert alle LEDs des TM FAST über SPI. [Guillaume: Specification for TM Fast]

Das **Frontend-Gerät** bildet den Kern des TM FAST. Hier wird die anwendungsspezifische Software abgelegt und ausgeführt. Das FPGA-Programm, bzw. die TM

FAST-Anwendung, besteht aus zwei Teilen, die FAST-Systemlogik (TFL FAST SYS) und der User-Logik (TFL FAST USER). Die Programmierung erfolgt in der Hardwarebeschreibungssprache Very High Speed Integrated Circuit Hardware Description Language (VHDL). Der Entwurf der Anwendungslogik ist in das System eingebettet und besitzt eine Schnittstelle zur Systemlogik in der Form der VHDL Top Entity TFL FAST USER e [Guillaume: Specification for TM Fast]. Über diese

können die Ein- und Ausgänge abgefragt bzw. gesteuert werden und die Kommunikation mit der CPU stattfinden. Ein VHDL-Programm besteht grundsätzlich aus den eingebundenen Packages bzw. Bibliotheken, der Entity, die als Schnittstellenbeschreibung fungiert, einer Architektur, die das Verhalten des Programmes beschreibt und einer Konfiguration, in der die entsprechende Architektur ausgewählt wird und gegebenenfalls Submodule und Parameter hinzugefügt werden [13]. Das spezifische Anwendungsprogramm wird somit in der VHDL-Architektur mit der Top Entity TFL FAST USER e implementiert. In Abb. 3 wird der Zusammenhang zwischen den beiden Logiken und die Einbindung in das System mit Backend-Gerät und Peripherie veranschaulicht.

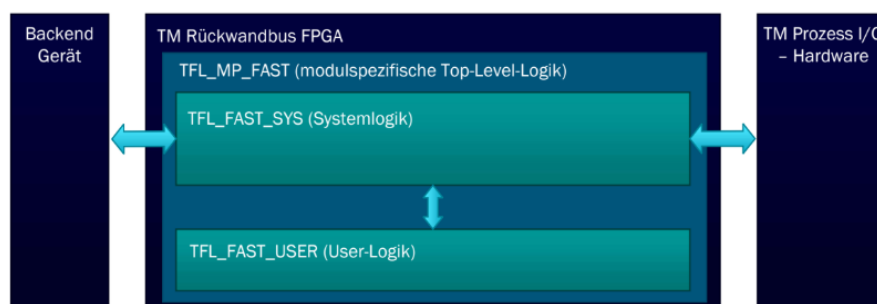


Abbildung 3.2: Abbildung 7: Aufbau aus Backend- und Frontend-Gerät und Zusammensetzung der Logik des TM FAST [Guillaume: Specification for TM Fast]

Als Frontend-Gerät wird das Intel® FPGA der Familie Cyclone 10 LP verwendet. Der exakte Gerätenamen lautet 10CL025YU256A7G. Die maximal verfügbaren **Ressourcen des FPGAs** sind in der nachfolgenden Tab. 1 aufgelistet.

Ressource	Anzahl
Logikelemente	24 624
M9K Memory Blöcke	66
18 x 18 Multiplizierer	66
PLL	4
Clocks	20
Maximale I/O	150

Tabelle 3.1: Ressourcen des Intel® FPGA Cyclone 10 LP [10]

Ein Logikelement (LE) ist die kleinste Logikeinheit in der Intel® Cyclone 10 LP

Gerätearchitektur. Jedes LE hat vier Eingänge, eine Look-Up Tabelle mit vier Eingängen, ein Register und eine Ausgangslogik. Die Look-Up Tabelle ist ein Funktionsgenerator, der jede Funktion mit vier Variablen implementieren kann. Der eingebettete Memory-Speicher ist aus M9K Memory-Blöcken aufgebaut. Jeder dieser Blöcke stellt 9 Kbit an On-Chip Speicher zur Verfügung. Diese M9K-Blöcke können beliebig miteinander verschachtelt und kombiniert werden, um größere Speicherplätze zu generieren. Insgesamt ergibt sich aus den 66 Blöcken eine Speicherkapazität von 594 Kbit, das umgerechnet 608256 Bit entspricht. Die Memory-Blöcke können als RAM, FIFO-Puffer oder ROM konfiguriert werden. Jeder eingebettete Multiplizierer-Block kann entweder als einzelner 18x18-Bit oder zwei individuelle 9x9-Bit Multiplizierer initialisiert werden. Die Funktionsweise der Blöcke kann entweder über die Parametrierung bestimmter IP-Cores in der Intel® Quartus® Prime Software definiert werden oder direkt über den VHDL oder Verilog HDL Code implementiert werden. Dem FPGA stehen vier Phasenregelschleifen (PLL) und bis zu 20 Clocks zur Verfügung. Die PLLs können dynamisch rekonfiguriert werden, wodurch die Phase und Frequenz der Clock geändert werden kann. Die General Purpose I/Os (GPIOs) des FPGAs bieten eine Vielzahl an Einstellmöglichkeiten, wie z.B. einen Pull-Up Widerstand oder eine programmierbare Anstiegsgeschwindigkeitsregelung zur Optimierung der Signalintegrität.[14] Nach Erläuterung des Aufbau- und Hardwarekonzepts des TM FAST folgt im restlichen Teil des Abschnittes der **äußere Aufbau** (siehe Abb. 4), die **Pinbelegung** der Klemmen, die die Anbindung von Peripheriegeräten ermöglichen sowie die genauere Erklärung der **Kommunikationsmöglichkeiten** zwischen FPGA und CPU. Das Modul besteht aus einer Baugruppe der Bauform MP, einem S7-1500 Frontstecker mit 40 Klemmen und vier zusätzlichen Klemmen zur Spannungsversorgung sowie einem TM FAST Debug Connector zur direkten Verbindung mit dem

FPGA. Dieser dient für die Durchführung von Service- und Entwicklungsarbeiten, unabhängig von der Frontstecker-Verdrahtung. Abb. 4 zeigt das Technologiemodul mit Zuordnung der äußeren Kennzeichnungen und Anzeigeeinheiten, die Erläuterung erfolgt dazu in Tab. 2. An der Außenseite kann man u.a. den Modultyp sowie -bezeichnung TM FAST, die Artikelnummer 6ES7554-1AA00-0AB0, den Funktionsstand und den Firmware-Stand V1.0.0 ablesen. Des Weiteren wird mit LEDs der Status der Ein- und Ausgänge, der Versorgungsspannung und der Diagnose angezeigt.

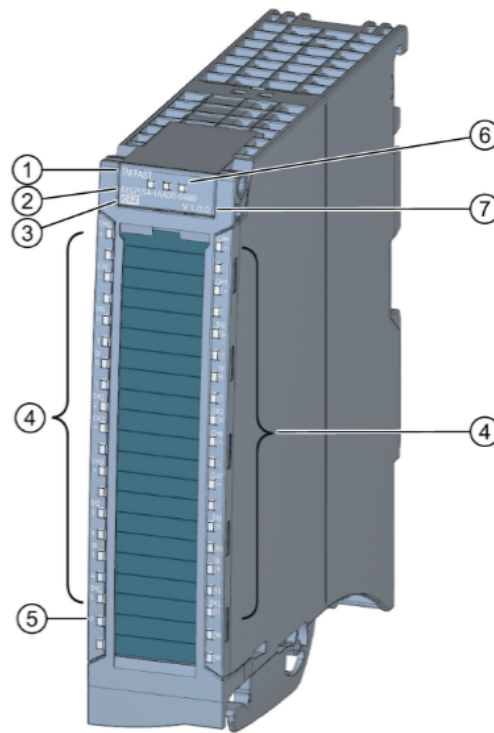


Abbildung 3.3: Abbildung 8: Technologiemodul TM FAST [3]

Num.	Bezeichnung
1	Modultyp und -bezeichnung
2	Artikelnummer
3	Funktionsstand
4	LEDs für Status der Ein- und Ausgänge
5	LED für Versorgungsspannung
6	LEDs für Diagnose
7	Firmware-Stand

Tabelle 3.2: Zuordnung der äußeren Kennzeichnungen und Anzeigeeinheiten gemäß Abb. 8

Um externe Signalgeber oder -empfänger mit dem TM FAST verbinden zu können, wird am Modul ein **40-poliger Frontstecker** angebracht. Dieser ist in Abb. 5 dargestellt. In Tab. 3.1 werden die jeweiligen Klemmen ihrer Funktion zugeordnet.

Das TM FAST umfasst insgesamt je acht 24 V-Digitaleingänge (DI) und -ausgänge (DQ) mit einem Ausgangsstrom von 0,1 A und einer maximalen Schaltfrequenz

von 200 kHz. Des Weiteren sind vier 24 V-Digitalein-/ausgänge (DIQ) mit 0,3 A Ausgangsstrom und maximal 1 kHz Schaltfrequenz verfügbar. Diese können in der Software als Ein- oder Ausgang konfiguriert werden. Die Digitalausgänge sind gegen Überlast und Kurzschluss geschützt.

Neben den digitalen Ein- und Ausgängen gibt es acht RS422/485-Kanäle (CH+ und CH-), die auch als TTL- (5 V) Kanäle eingesetzt werden können. Die Richtung ist wie bei den DIQ-Kanälen einstellbar. Wird mit einem RS485-Signal gearbeitet, so wird jeweils ein Leitungspaar verwendet. Bei einem Signal mit TTL-Standard wird nur eine einzelne Leitung benötigt.

Um kurze Störungen oder instabile Signale auf der Leitung zu filtern, kann für die Digitaleingänge und Kanäle eine Eingangsverzögerung parametrisiert werden. Den DIs kann eine Eingangsverzögerung im Bereich von 0 bis 20 ms in Schritten von 0,001 ms fest in den Systemlogik-Parametern eingestellt werden. Die gleiche Möglichkeit besteht für die RS422/RS485/TTL-Kanäle. Hier kann die Eingangsverzögerung im Bereich von 0,1 μ s bis 1000 μ s festgelegt werden.

Neben der festen Parametrierung kann die Verzögerung auch in der TM FAST-Anwendung frei programmiert werden [3].

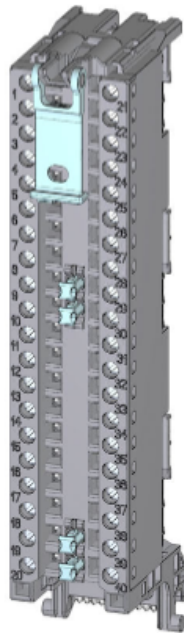


Abbildung 3.4: Frontstecker des TM FAST [9]

Klemmen	Anschlussbelegung
1..4, 10..13, 21..24, 30..33	RS422/RS485/TTL-Ein- oder Ausgangssignal
5, 6, 14, 15, 25, 26, 34, 35	Digitalausgang
7, 8, 16, 17, 27, 28, 36, 37	Digitaleingang
9, 18, 29, 38	Digitalein-/ausgang
19, 20, 39, 40	Masse

Tabelle 3.3: Anschlussbelegung des Frontsteckers aus Abb. 9

Die **Kommunikation** zwischen CPU und TM FAST kann entweder zyklisch oder azyklisch erfolgen. Die zyklischen Ein- und Ausgangsdaten sind unterteilt in acht Doppelwörter. Ein Doppelwort besteht aus 32 Bit, sodass in Summe mit 32 Byte kommuniziert werden kann. Der zyklische Datenaustausch wird in der Regel aus der Sicht der CPU betrachtet. Das bedeutet, dass die Steuerschnittstelle (CTRL IF) die Ausgangsdaten, die von der CPU an das TM gesendet werden, beinhaltet. Entsprechend werden die Eingangsdaten, die die CPU aus dem TM FAST liest, als Rückmeldeschnittstelle (FB IF) bezeichnet. Auf Seiten des TMs ist die Verwendung der Schnittstellen folglich genau entgegengesetzt. In der Umsetzung der Laser-Scanner-Anwendung in Abschnitt 3.1 erfolgt der Datenaustausch rein über die zyklische Schnittstelle zwischen CPU und Technologiemodul. Der Grund hierfür ist, dass somit der Zeitpunkt des Datentransfers genau bestimmt werden kann und mit jedem Zyklus der CPU neue Werte übermittelt werden können. Die azyklische Datenmenge kann für eine Anwendungslogik konfiguriert werden. Diese ist dann für diese Logikversion fest hinterlegt und kann während der Ausführung nicht mehr geändert werden. Es kann zwischen 4, 32, 64 und 128 Byte für die azyklische Kommunikation gewählt werden. Es steht je ein Register mit der gewählten Größe für den Lese- und Schreibprozess zur Verfügung. Der Datenaustausch erfolgt über die Anweisungen LTMFAST UserWriteREC (Schreibzugriff) und LTMFAST UserReadREC (Lesezugriff) im TIA Portal (Abschnitt 2.3.1). Es können verschiedene Parameter genutzt werden, die den Ablauf des Informationsaustausches koordinieren und steuern. [[?], [3]]

3.2 Funktionsweise des FM350-2

Die Funktionsbaugruppe FM 350-2 ist eine 8-kanalige Zählerbaugruppe mit integrierter Dosierfunktion für das Automatisierungssystem S7-300. Sie wird vor allem in Bereichen eingesetzt, in denen Signale gezählt und schnelle Reaktionen auf bestimmte Zählerstände notwendig sind. Besonders relevant ist die FM 350-2 für Anwendungen in der Fertigungs- und Automatisierungstechnik, wie beispielsweise in Verpackungs-, Sortier- und Dosieranlagen sowie bei der Drehzahlregelung und Überwachung von Gasturbinen.

Sie unterstützt einen maximalen Zählbereich von -2^{31} bis $+2^{31} - 1$ (entsprechend $-2\,147\,483\,648$ bis $+2\,147\,483\,647$) und erreicht je nach verwendeter Gebersignale eine maximale Eingangsfrequenz von bis zu 20 kHz pro Kanal [2].

Die Baugruppe ermöglicht verschiedene Betriebsarten: endloses, einmaliges sowie periodisches Zählen (jeweils vorwärts oder rückwärts), Frequenzmessung, Drehzahlmessung, Periodendauermessung sowie präzises Dosieren. Der Zählvorgang kann wahlweise über das Anwenderprogramm (Software-Tor) oder über externe Signale (Hardware-Tor) gestartet und gestoppt werden. Dabei können Zähl-, Tor- und Richtungssignale direkt an die Baugruppe angeschlossen werden.

Für jeden Zählkanal ist es möglich, einen Vergleichswert zu definieren. Im Dosiermodus lassen sich bis zu vier Vergleichswerte je Kanal verwenden. Wird der definierte Vergleichswert erreicht, kann automatisch ein zugeordneter Ausgang gesetzt oder zurückgesetzt werden, um direkt steuerungsrelevante Aktionen auszulösen oder Prozessalarme zu generieren.

Innerhalb der Betriebsarten „Einmalig zählen“, „Periodisch zählen“ und „Dosieren“ lassen sich zusätzlich Zählgrenzen innerhalb des maximalen Zählbereichs festlegen. Im Vorwärtsbetrieb beginnt die Zählung bei 0, der Endwert ist dabei frei wählbar. Im Rückwärtsbetrieb wird der Startwert definiert, während der Endwert bei 0 liegt.

Die FM 350-2 unterstützt pro Zählkanal bis zu vier Prozessalarme. Zwei davon können durch Flankenwechsel am Hardware-Tor ausgelöst werden, zwei weitere sind abhängig von der jeweils eingestellten Betriebsart. Im Dosierbetrieb können bis zu fünf spezifische Alarmerzeuger definiert werden. (Wichtig welche + Arten?)

Für die Zählerfunktion können Signale unterschiedlicher Gebertypen verwendet werden. Zulässig sind ausschließlich prellfreie Geber, darunter 24-V-Inkrementalgeber (Gegentakt oder P-Schalter), 24-V-Impulsgeber mit Richtungspegel, 24-V-Initiatoren ohne Richtungspegel (z. B. Lichtschranken oder BERO Typ 2) sowie NAMUR-Geber nach DIN 19234. Die Signaltypen können an den Eingängen in Vierergruppen zwischen 24-V- und NAMUR-Signalen konfiguriert werden. Zu beachten ist, dass an für NAMUR konfigurierte Eingänge keine Signalspannungen über 8,2 V angelegt werden dürfen. An den Tor- (logische Freigabeeingänge, die den Zählpfad öffnen oder sperren) und Richtungseingängen sind ausschließlich 24-V-Signale erlaubt.

Zum Schutz vor Störungen sind alle Eingänge mit einem einheitlichen Eingangsfilter (RC-Glied) mit einer Filterzeit von 50 μ s versehen. Schnelle Reaktionen auf Zählereignisse sind durch digitale Ausgänge realisierbar, wobei pro Kanal ein Ausgang und im Dosiermodus bis zu vier Ausgänge verfügbar sind. Diese können zählerstandsabhängig oder über Steuerbits angesteuert werden.

Das Verhalten der Baugruppe bei einem CPU-STOP ist parametrierbar. Es kann festgelegt werden, ob die Betriebsart fortgeführt oder abgebrochen wird und ob die digitalen Ausgänge ihren aktuellen Zustand beibehalten, auf Ersatzwerte gesetzt oder abgeschaltet werden. Dabei ist besondere Vorsicht geboten, da Ersatzwerte auch an nicht freigegebenen Ausgängen anliegen können, was unter Umständen zu gefährlichen Anlagenzuständen führt.

Auch bei Ausfall der Baugruppenversorgung zeigt die FM 350-2 ein differenziertes Verhalten. Wird sie über einen Standard-Rückwandbus betrieben, erkennt die CPU bei Spannungsverlust einen Peripherie-Zugriffsfehler, und die Baugruppe startet nach Spannungswiederkehr nicht automatisch neu. Beim Einsatz eines aktiven Rückwandbusses hingegen wird der CPU ein Ziehen-Alarm und bei Wiederkehr der Versorgungsspannung ein Stecken-Alarm gemeldet.

Zusätzlich verfügt die FM 350-2 über umfassende Diagnosefunktionen. Mögliche Fehlerzustände wie fehlerhafte Geberversorgung (NAMUR), fehlende oder fehlerhafte Parametrierung, Zeitüberwachungsfehler (Watchdog), Drahtbruch oder Kurzschluss sowie der Verlust von Prozessalarmen können erkannt und gemeldet werden [2].

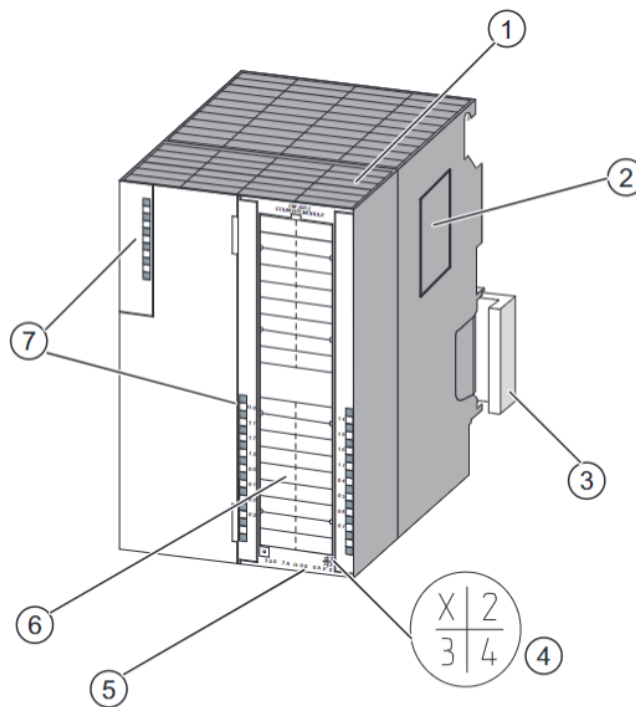


Abbildung 3.5: Hardware der FM-350-2 [2]

Nummer	Bezeichnung
1	Frontstecker
2	Typenschild
3	Busverbinder SIMATIC-Schnittstelle
4	Erzeugnisstand
5	Bestellnummer
6	Beschriftungsstreifen
7	Diagnose-LED
8	Status-LEDs

Tabelle 3.4: Kennzeichnungen und Komponenten der FM 350-2

3.2.1 Endloses Zählen bei der FM 350-2

Die Betriebsart „Endloses Zählen“ ermöglicht es der FM 350-2, Zählimpulse kontinuierlich zu erfassen, ohne dass der Zähler bei Erreichen einer Grenze stoppt. Wird beim Vorwärtzzählen die obere Zählgrenze (+2147483647) erreicht und

ein weiterer Zählimpuls empfangen, springt der Zähler automatisch auf die untere Zählgrenze ($-2\,147\,483\,648$) und zählt weiter. Analog dazu springt der Zähler beim Rückwärtszählen von der unteren Zählgrenze auf die obere Zählgrenze, sobald ein weiterer Zählimpuls erfolgt. Dadurch wird ein endloses Zählen innerhalb des 32-Bit-Zählbereichs realisiert.

Der Zählbereich ist fest auf 31 Bit vorzeichenbehaftet ($-2\,147\,483\,648$ bis $+2\,147\,483\,647$) und kann nicht verändert werden. Nach einem Neustart beginnt der Zähler bei 0.

Wurde ein Vergleichswert parametrierbar, kann bei Erreichen des Vergleichswerts ein Prozessalarm ausgelöst und/oder ein Ausgang geschaltet werden.

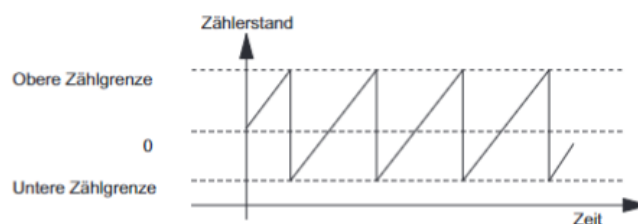


Abbildung 3.6: Endloses Zählen in Hauptzählrichtung vorwärts [2]

3.2.2 Einmaliges Zählen bei der FM 350-2

Die Betriebsart „Einmaliges Zählen“ ermöglicht das Erfassen einer festen Anzahl von Zählimpulsen zwischen einem definierten Start- und Endwert. Über die Parametrierung werden Startwert, Endwert und Hauptzählrichtung festgelegt.

Beim Zählen in Hauptzählrichtung vorwärts beginnt der Zähler bei 0 und zählt einmalig bis zum Endwert. Wird der Wert „Endwert-1“ erreicht und ein weiterer Zählimpuls empfangen, springt der Zähler zurück auf den Zählerstand 0 und bleibt dort stehen, auch wenn weitere Impulse folgen.

Beim Zählen in Hauptzählrichtung rückwärts startet der Zähler beim Startwert und zählt einmalig in Richtung 0. Erreicht der Zähler den Wert 1 und ein weiterer Zählimpuls erfolgt, springt er auf den Startwert zurück und bleibt stehen.

Wird entgegen der gewählten Hauptzählrichtung gezählt und dabei der Startwert unter- oder überschritten, meldet die Baugruppe den aktuellen Zählerstand mit negativem Vorzeichen zurück. Ein Überlauf oder Unterlauf findet in diesem Modus nicht statt.

Ist ein Vergleichswert parametrierbar, kann bei aktuellem Zählerstand = Vergleichswert ein Prozessalarm ausgelöst und/oder ein Ausgang geschaltet werden.

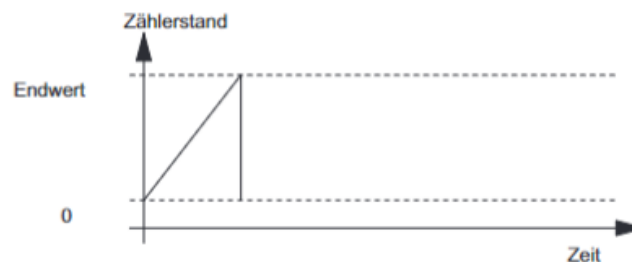


Abbildung 3.7: Einmaliges Zählen in Hauptzählrichtung vorwärts [2]

3.2.3 Periodisches Zählen bei der FM 350-2

Die Betriebsart „Periodisches Zählen“ erlaubt das Erfassen von Zählimpulsen innerhalb eines festgelegten Bereichs zwischen Startwert und Endwert. Nach Erreichen des Endwerts springt der Zähler automatisch auf den Startwert zurück und zählt erneut bis zum Endwert. Dieses Verhalten wiederholt sich zyklisch, wodurch eine periodische Zählung realisiert wird.

Beim Zählen in Hauptzählrichtung vorwärts startet der Zähler beim Startwert (meist 0) und zählt bis zum Endwert. Sobald der Endwert erreicht ist und ein weiterer Zählimpuls erfolgt, springt der Zähler zurück auf den Startwert und beginnt erneut zu zählen. Die Zählimpulse werden kontinuierlich erfasst, sodass der Zählvorgang periodisch erfolgt.

Beim Zählen in Hauptzählrichtung rückwärts startet der Zähler am parametrierbaren Startwert und zählt in Richtung Endwert (meist 0). Wird der Endwert erreicht und ein weiterer Zählimpuls empfangen, springt der Zähler auf den Startwert zurück und zählt erneut rückwärts.

Wird entgegen der gewählten Hauptzählrichtung gezählt und dabei der Startwert unter- oder überschritten, meldet die Baugruppe den aktuellen Zählerstand mit negativem Vorzeichen zurück. Ein Überlauf oder Unterlauf findet in diesem Modus nicht statt; der Ausgangswert bleibt unverändert.

Ist ein Vergleichswert parametrierbar, kann bei aktuellem Zählerstand = Vergleichswert ein Prozessalarm ausgelöst und/oder ein Ausgang geschaltet werden.

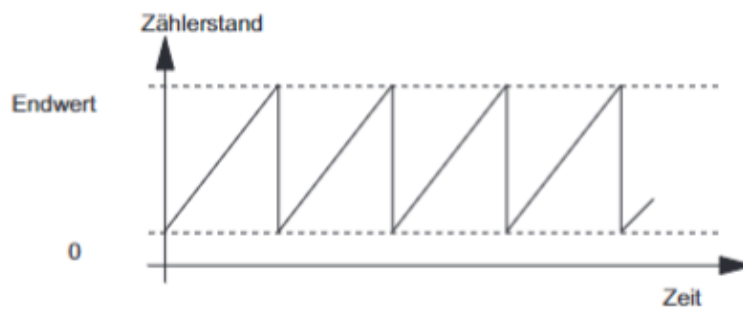


Abbildung 3.8: Periodisches Zählen in Hauptzählrichtung vorwärts [2]

3.2.4 Frequenzmessung mit der FM 350-2

Die FM 350-2 bietet eine integrierte Funktion zur Frequenzmessung, bei der die Anzahl der Impulse innerhalb eines parametrierbaren Zeitfensters gezählt wird. Die Zeitfensterlänge kann zwischen 10 ms und 10 s eingestellt werden. Nach Ablauf des Zeitfensters wird die Frequenz als Anzahl der gezählten Impulse pro Sekunde berechnet und als Wert in Hz ausgegeben.

Die Frequenzmessung kann sowohl über das Anwenderprogramm als auch über externe Signale (Hardware-Tor) gestartet und gestoppt werden. Die Baugruppe unterstützt dabei verschiedene Vergleichswerte, sodass Prozessalarme ausgelöst werden können, wenn die gemessene Frequenz bestimmte Grenzwerte über- oder unterschreitet.

Die wichtigsten Merkmale der Frequenzmessung sind:

- Messbereich: 0 Hz bis 9.999.999 Hz (abhängig von Parametrierung und Signalart)
- Einstellbare Zeitfenster für die Messung (10 ms bis 10 s)
- Prozessalarme bei Über- oder Unterschreiten von Grenzwerten
- Start und Ende der Messung über Software- oder Hardware-Tor
- Frequenzvergleich mit parametrierten Sollwerten möglich

Die Frequenzmessung eignet sich besonders für Anwendungen, bei denen die Geschwindigkeit oder Drehzahl von Maschinen und Anlagen überwacht werden muss. Typische Einsatzfälle sind die Überwachung von Förderbändern, Drehtischen oder Antrieben.

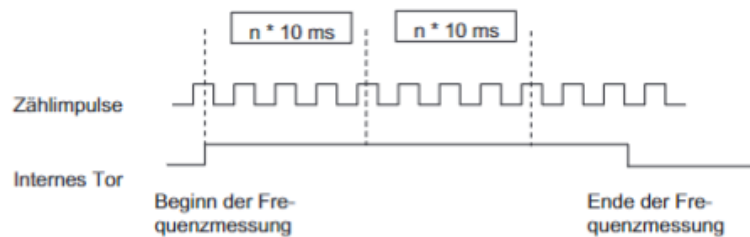


Abbildung 3.9: Prinzip der Frequenzmessung mit Zeitfenster bei der FM 350-2 [2]

3.2.5 Drehzahlmessung mit der FM 350-2

Die Drehzahlmessung ist eine spezielle Betriebsart der FM 350-2, die nahezu identisch mit der Frequenzmessung arbeitet. Neben der Länge des Zeitfensters muss bei der Drehzahlmessung in der Parametermaske die Anzahl der Impulse pro Motor- oder Geberumdrehung angegeben werden.

Am Ende jedes Zeitfensters wird der Drehzahlwert aktualisiert. Die ermittelte Drehzahl wird in der Einheit 1×10^{-3} Umdrehungen/Minute ausgegeben.

Wird kein gültiger Wert ermittelt, wird -1 zurückgemeldet. Werden in einem Zeitintervall keine Impulse gezählt, liefert die Baugruppe 0×10^{-3} U/min ($= 0$ U/min).

Über zwei Drehzahlvergleichswerte (unterer und oberer Grenzwert) kann überwacht werden, ob die gemessene Drehzahl im vorgegebenen Bereich liegt. Beim Verlassen dieses Bereichs kann ein Prozessalarm ausgelöst werden. Die FM 350-2 prüft, ob Drehzahlobergrenze $>$ Drehzahluntergrenze ist und meldet einen Parametrierungsfehler, falls dies nicht der Fall ist.

Die Drehzahlmessung wird über die Torfunktionen gestartet und beendet. Folgende Prozessalarme sind möglich:

- Beginn der Drehzahlmessung durch HW-Tor (positive Flanke)
- Ende der Drehzahlmessung durch HW-Tor (negative Flanke)

- Ende der Messwerterfassung (Integrationszeit ist abgelaufen)
- Über- bzw. Unterschreiten der Drehzahlgrenzen

Die Drehzahlmessung eignet sich besonders für Anwendungen, bei denen die Drehzahl von Maschinen oder Antrieben überwacht und geregelt werden muss, wie z.B. bei Förderbändern, Drehtischen oder Motoren.

3.2.6 Periodendauermessung mit der FM 350-2

Bei sehr kleinen Frequenzen wird anstelle der Frequenz die Periodendauer gemessen. Im Betriebsart „Periodendauermessung“ erfasst die FM 350-2 die exakte Zeit zwischen zwei steigenden Flanken eines Eingangssignals. Die Messung wird über Torsignale (Hardware- oder Software-Tor) gestartet und beendet.

Die Periodendauer kann nur in der eingestellten Hauptzählrichtung erfasst werden. Der zulässige Messbereich liegt zwischen 40 μs und 120 s (entspricht 25 000 Hz bis 0,00833 Hz). Liegt kein gültiger Wert vor, wird -1 zurückgemeldet.

Über die Parametermaske können zwei Periodendauer-Grenzwerte eingestellt werden:

- Untere Grenze: 0 μs bis 119 999,999 μs
- Obere Grenze: 40 μs bis 120 000,000 μs

Folgende Prozessalarme sind möglich:

- Beginn der Periodendauermessung durch HW-Tor (positive Flanke)
- Ende der Periodendauermessung durch HW-Tor (negative Flanke)
- Ende der Messwerterfassung (Integrationszeit ist abgelaufen)
- Über- bzw. Unterschreiten der Periodendauergrenzen

Die Periodendauermessung eignet sich besonders für Anwendungen mit sehr niedrigen Frequenzen, bei denen eine präzise Zeitmessung zwischen zwei Ereignissen erforderlich ist, z.B. bei langsamen Bewegungen oder Taktgebern.

3.2.7 Dosierfunktion der FM 350-2

Die FM 350-2 bietet eine integrierte Dosierfunktion, mit der präzise Mengen von Materialien oder Teilen gesteuert und abgegeben werden können. Im Betriebsmodus „Dosieren“ werden jeweils vier Zählkanäle zu einem Dosierkanal zusammengefasst. Für jeden Dosierkanal können bis zu vier Vergleichswerte festgelegt werden, die einzeln oder gruppenweise verändert werden können.

Der aktuelle Zählerstand wird kontinuierlich mit den Vergleichswerten verglichen. Sobald der Zählerstand einen Vergleichswert erreicht, kann ein zugeordneter Digitalausgang angesteuert und/oder ein Prozessalarm ausgelöst werden. So lassen sich bis zu vier Dosierabschnitte innerhalb eines Dosierzyklus realisieren.

Die Dosierfunktion unterstützt sowohl den Vorwärts- als auch den Rückwärtsbetrieb. Im Rückwärtsbetrieb wird ein Startwert vorgegeben und der Zähler zählt bis zum Endwert (meist 0) herunter. Die Vergleichswerte definieren die einzelnen Dosierabschnitte, in denen jeweils ein Ausgang geschaltet oder ein Alarm ausgelöst werden kann.

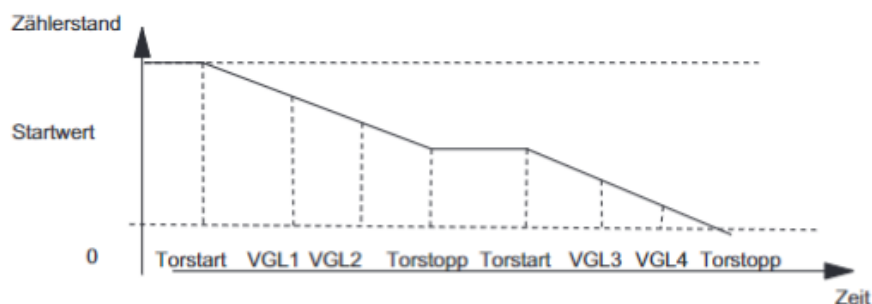


Abbildung 3.10: Dosierfunktion: Hauptzählrichtung rückwärts mit vier Vergleichswerten und Prozessalarmen [2]

Mögliche Prozessalarme im Dosierbetrieb sind:

- Beginn der Dosierung durch Hardware-Tor (positive Flanke)
- Abbruch/Unterbrechung der Dosierung durch Hardware-Tor (negative Flanke)
- Prozessalarm für jeden Vergleichswert (bis zu vier pro Kanal)
- Erreichen der Zählbereichsgrenzen (Endwert/Startwert)

Die Dosierfunktion eignet sich besonders für Anwendungen, bei denen exakte Mengen abgegeben werden müssen, wie z.B. in Abfüllanlagen, Mischprozessen oder bei der Steuerung von Förderbändern. Durch die flexible Parametrierung der Vergleichswerte und die schnelle Reaktion auf Zählereignisse können komplexe Dosieraufgaben zuverlässig und präzise umgesetzt werden.

3.2.8 Torfunktionen der FM 350-2

Viele Anwendungen erfordern, dass der Zählvorgang erst zu einem definierten Zeitpunkt, abhängig von anderen Ereignissen, gestartet oder gestoppt wird. Dieses Starten und Stoppen des Zählvorgangs geschieht bei der FM 350-2 über eine Torfunktion. Wird das Tor geöffnet, können Zählimpulse zum Zähler gelangen, der Zählvorgang wird fortgesetzt. Wird das Tor geschlossen, können keine Zählimpulse mehr zum Zähler gelangen, der Zählvorgang ist gestoppt.

Software-Tor und Hardware-Tor Die FM 350-2 besitzt zwei Torfunktionen:

- **Software-Tor (SW-Tor):** Das Software-Tor wird durch das Steuerbit `SW_GATE` im Anwenderprogramm gesetzt oder zurückgesetzt. Es kann ausschließlich durch einen Flankenwechsel von 0 auf 1 geöffnet und von 1 auf 0 geschlossen werden.
- **Hardware-Tor (HW-Tor):** Das Hardware-Tor wird durch ein externes Signal an einem digitalen Eingang der Baugruppe gesteuert. Es wird durch einen Flankenwechsel des Eingangssignals geöffnet oder geschlossen.

Internes Tor Das interne Tor ergibt sich als logische UND-Verknüpfung von HW-Tor und SW-Tor. Ist eines der Tore geschlossen, bleibt das interne Tor ebenfalls geschlossen und der Zählvorgang ist inaktiv. Nur wenn beide Tore offen sind, ist das interne Tor aktiv und der Zählvorgang läuft.

HW-Tor	SW-Tor	internes Tor	Zählvorgang
offen	offen	offen	aktiv
offen	geschlossen	geschlossen	inaktiv
geschlossen	offen	geschlossen	inaktiv
geschlossen	geschlossen	geschlossen	inaktiv

Tabelle 3.5: Zusammenhang der Torfunktionen und des Zählvorgangs

Beispiel Mit dem Setzen des Torsignals wird das Tor geöffnet und die Zählimpulse werden gezählt. Wird das Torsignal weggenommen, wird das Tor geschlossen und die Zählimpulse werden nicht mehr vom Zähler erfasst. Der Zählerstand bleibt konstant.

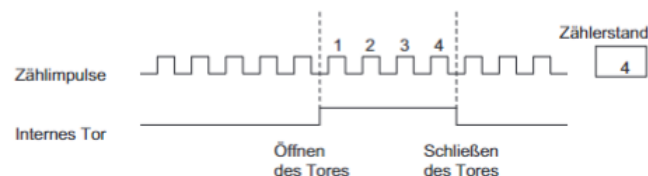


Abbildung 3.11: Öffnen und Schließen eines Tores und das Zählen der Impulse [2]

3.2.9 Prozessalarme der FM 350-2

Im Betrieb der Zählerbaugruppe FM 350-2 können verschiedene Fehler auftreten. Diese entstehen z. B. durch Defekte auf der Baugruppe, fehlerhafte Bedienung/-Verdrahtung oder widersprüchliche Parametrierung. Zur Erkennung und gezielten Reaktion unterscheidet die Baugruppe Fehlerklassen und löst bei Bedarf Prozessalarme aus.

Fehlerklassen und -arten

- Interner Fehler: Defekte oder fehlerhafte Zustände innerhalb der Baugruppe, z. B. Ansprechen der Zeitüberwachung (Watchdog).
- Externer Fehler: Probleme in der Peripherie außerhalb der Baugruppe, die keinem einzelnen Kanal zugeordnet werden können.

- Externer Kanalfehler: Kanalspezifische Fehler, z. B. Signalfehler durch einen NAMUR-Geber.
- Datenfehler: Unzulässige Aufträge von Automatisierungsgerät (AG) oder Programmiergerät (PG), z. B. Vergleichswerte außerhalb des Zählbereichs.

Reaktion der Baugruppe

- Interne, externe und externe Kanalfehler: Sammelfehler-LED (SF) leuchtet; es können Diagnosealarme ausgelöst werden (wenn freigegeben).
- Datenfehler: Auftrag wird abgelehnt; Eintrag im Diagnosepuffer; Behebung durch erneute Übertragung mit korrigierten Daten.

Sammelfehler-LED (SF) Leuchtet rot bei internem Baugruppenfehler, fehlerhafter Parametrierung oder Leitungsfehlern. Typische Ursachen: angesprochene Zeitüberwachung, verlorener Prozessalarm, fehlende/fehlerhafte Parametrierung, Probleme der Geberversorgung oder fehlerhafte NAMUR-Gebersignale (z. B. Drahtbruch/Kurzschluss).

Diagnosealarme und -daten Diagnosealarme können in der Parametrierung freigegeben werden. Beim Fehler ruft die CPU den Diagnose-OB OB 82 auf; das zyklische Programm wird unterbrochen.

- Diagnosedaten DS0: werden beim Aufruf von OB 82 automatisch übertragen.
- Diagnosedaten DS1: detaillierte Kanalinfos, auslesbar über FC DIAG_RD.
- Inhalt: Hinweise zu intern/extern/Parametrier-/Kanalfehlern, Ansprechen der Zeitüberwachung, verlorene Prozessalarme, Leitungs-/Geberversorgungsfehler.
- Zusätzlich können über SFC 52 benutzerdefinierte Meldungen in den Diagnosepuffer der CPU geschrieben werden.

Datenfehler und Quittierung Ergeben sich aus unzulässigen Aufträgen durch PG oder über FC CNT2_WR bzw. FB CNT2WRPN. Die Baugruppe prüft und lehnt fehlerhafte Aufträge ab; das Bit DATA_ERR im Zähler-Datenbaustein wird gesetzt. Behebung durch Korrektur und erneute Übertragung des Auftrags. Anzeige im Diagnosepuffer sowie im Menü Test > Fehlerevaluation.

3.3 Direkter Vergleich der TM Fast und FM 350-2

Kriterium	TM FAST	FM 350-2
Unterschiede		
Architektur	Intel Cyclone 10 LP FPGA, frei programmierbar (VHDL), Backend/Frontend-Konzept	Feste ASIC-basierte Zählerbaugruppe, keine freie Hardwareprogrammierung
Zählerkanäle	Bis zu 14 Kanäle (DI, DQ, DIQ, RS422/485, TTL), flexibel konfigurierbar	8 Kanäle, fest als Zähler ausgelegt
Zählsignale / Gebereingänge	24 V-DI/DIQ, RS422/485, TTL, flexibel konfigurierbar, max. 5 MHz (DIQ/RS422/485), 200 kHz (DI/DQ)	Namur (DIN 19234), 24 V Impulsgeber, Inkrementalgeber, Richtungssignal, max. 20 kHz (A0...7), 10 kHz (B0...7)
Digitaleingänge	8x 24 V-DI, 4x 24 V-DIQ (konfigurierbar), max. 200 kHz (DI), 1 kHz (DIQ), Eingangsverzögerung parametrierbar (0–20 ms)	8x 24 V-DI, 0-Signal: -3 bis 5 V, 1-Signal: 11–30,2 V, Eingangsstrom: 0-Signal ≤ 2 mA, 1-Signal 9 mA, Eingangsverzögerung max. 50 μ s
Digitalausgänge	8x 24 V-DQ (0,1 A, 200 kHz), 4x DIQ (0,3 A, 1 kHz), High-/Low-aktiv, Überlast-/Kurzschlusschutz	Bis zu 4 Ausgänge pro Kanal (Dosiermodus), 24 V, direkt an Zählerereignisse gekoppelt, Verhalten parametrierbar
Reaktionszeit	< 1 μ s (laut Gerätehandbuch)	Nicht explizit angegeben
Zählbereich	32 Bit, flexibel nutzbar	32 Bit, fest (-2^{31} bis $2^{31} - 1$)

Betriebsarten	Frei programmierbar (z.B. Rundenzähler, Pulsfolgen, Dosierlogik, benutzerdefiniert)	Endlos, einmalig, periodisch, Dosieren, Frequenz-, Drehzahl-, Periodendauermessung
Parametrierung	Azyklisch und im laufenden Betrieb über UserWriteREC, flexible Steuerung	Parametrierung über STEP 7, Änderungen meist nur im Stopp-Betrieb
Ausgangslogik	Pro Kanal High-/Low-aktiv, Steuerbits, Rückmeldung, flexible Reaktion	Feste Zuordnung, Ausgänge direkt an Zählerereignisse gekoppelt
Diagnose	Umfangreiche Diagnosemöglichkeiten, frei programmierbar	Umfassende Diagnose, fest integriert (z.B. SF-LED, OB82, Diagnosedaten)
Integration	S7-1500, TIA Portal, flexible Einbindung, Peripherieanbindung	S7-300, STEP 7, klassische Integration
Gemeinsamkeiten		
Zählfunktionen	Hohe Zählauflösung (32 Bit), schnelle Verarbeitung, Einsatz als Zähler in der Automatisierungstechnik	
Frequenz-/Drehzahlmessung	Unterstützung von Frequenz- und Drehzahlmessung, Prozessüberwachung	
Industrielle Anwendungen	Verpackung, Sortierung, Dosierung, Maschinenbau, Prozessautomatisierung	
Diagnose	Umfangreiche Diagnose- und Fehleranzeigen, Unterstützung von Alarmen	

Tabelle 3.6: Vergleich der Eigenschaften von TM FAST und FM 350-2, insbesondere Zählsignale und Digitalein-/ausgänge (Zeitdaten TM FAST gemäß [3, 9], FM 350-2 gemäß Handbuch)

4 Entwicklung des VHDL-basierten Applikationsbeispiels

4.1 Anforderungen an die Neuentwicklung

Für die Neuentwicklung des Zählermoduls auf Basis des TM FAST ergeben sich verschiedene Anforderungen, die sich aus den funktionalen und technischen Zielen ableiten. Das Modul soll mindestens acht unabhängige Zählerkanäle bereitstellen, die flexibel als Digitaleingänge, Digitalausgänge oder RS485/TTL-Kanäle genutzt werden können. Jeder Kanal arbeitet mit einer ausreichend breiten Zählvariable, typischerweise 32 Bit, um auch große Wertebereiche abzudecken. Die Übergabe von Sollwerten und Parametern muss effizient und im laufenden Betrieb möglich sein, sodass der Prozess nicht unterbrochen wird. Verschiedene Betriebsmodi wie Rundenzähler, Reset, Hold und Start/Stop-Funktionen sind zu unterstützen. Für jeden Kanal werden Statusinformationen und Prozesswerte, etwa der aktuelle Zählerstand oder die Geschwindigkeit, rückgemeldet. Fehler müssen erkannt, sicher behandelt und an die Steuerung gemeldet werden. Die Lösung ist so zu gestalten, dass sie vollständig mit dem TIA Portal integrierbar und zur FM350-2 kompatibel ist. Zudem ist auf eine ressourcenschonende Implementierung zu achten, um FPGA-Ressourcen effizient zu nutzen und eine einfache Erweiterbarkeit zu ermöglichen.

Die konkrete Umsetzung dieser Anforderungen wird in den folgenden Abschnitten detailliert beschrieben.

4.2 Modularisierung und Wiederverwendbarkeit

Die Architektur des VHDL-Designs ist konsequent modular aufgebaut und nutzt Generics, Arrays und Generate-Statements, um eine hohe Wiederverwendbarkeit

und Erweiterbarkeit zu gewährleisten. Die Anzahl der Kanäle wird über das Generic ‘CHANNEL_COUNT‘ zentral festgelegt. Für alle kanalbezogenen Signale und Zustände werden Array-Typen verwendet, sodass die Logik für jeden Kanal identisch und unabhängig voneinander instanziiert werden kann.

Durch die Verwendung von Generate-Blöcken (‘channel_gen : for i in 0 to CHANNEL_COUNT-1 generate‘) wird die komplette Zähler- und Frequenzmesslogik pro Kanal automatisch erzeugt. Neue Kanäle können einfach durch Erhöhen von ‘CHANNEL_COUNT‘ und Anpassen der Mapping-Arrays hinzugefügt werden, ohne dass das Gesamtdesign oder die Prozesslogik angepasst werden muss.

Auch die Kanaluordnung (z.B. für unterschiedliche Hardware-Konfigurationen) erfolgt über Mapping-Arrays (‘CHANNEL_MAP_IN_CH‘, ‘CHANNEL_MAP_OUT_CH‘), die flexibel anpassbar sind. Die Modularisierung erleichtert zudem die Wartung, das Testen und die Erweiterung um neue Funktionen, wie zusätzliche Diagnosemechanismen oder alternative Betriebsmodi, ohne tiefgreifende Änderungen an der bestehenden Architektur.

4.3 Flexibilität der Kanaluordnung

Das VHDL-Design nutzt spezielle Mapping-Arrays, um eine flexible Zuordnung der Ein- und Ausgänge zu den einzelnen Kanälen zu ermöglichen. Die Arrays ‘CHANNEL_MAP_IN_CH‘ und ‘CHANNEL_MAP_OUT_CH‘ legen für jeden logischen Kanal fest, welcher physikalische Eingang bzw. Ausgang verwendet wird. Dadurch kann das Design einfach an unterschiedliche Hardware-Konfigurationen oder Kundenanforderungen angepasst werden, ohne dass die eigentliche Kanal-Logik verändert werden muss.

Im Code werden diese Arrays wie folgt deklariert und verwendet:

```

1 type t_channel_map is array (0 to 13) of integer;
2 constant CHANNEL_MAP_IN_CH : t_channel_map := (0, 1, 3, 4, 6, 7, 9, 10
3 constant CHANNEL_MAP_OUT_CH : t_channel_map := (0, 1, 3, 4, 6, 7, 9, 10

```

Jeder Kanal greift über diese Mapping-Arrays auf die korrekten Ein- und Ausgänge zu:

```

1 input_selector(i) <= RS485_RX(CHANNEL_MAP_IN_CH(i)) when i > 8 else DI(O
2 ...
3 DQ(CHANNEL_MAP_OUT_CH(i)) <= out_val;

```

Dadurch ist es möglich, die Kanalbelegung zentral zu ändern, etwa um bestimmte Kanäle als RS485, Digitaleingang oder Digitalausgang zu nutzen. Auch bei Änderungen der Hardware (z.B. andere Verdrahtung oder Erweiterung der Kanäle) muss lediglich das Mapping angepasst werden, ohne dass die restliche Kanal- oder Prozesslogik betroffen ist. Dies erhöht die Wartbarkeit und Wiederverwendbarkeit des Designs erheblich und erlaubt eine schnelle Anpassung an neue Anforderungen.

4.4 Ressourcenoptimierung

Das VHDL-Design nutzt gezielt Arrays und Bitfelder, um den Ressourcenverbrauch im FPGA zu minimieren. Die Anzahl der tatsächlich verwendeten Kanäle wird zentral über das Generic ‘CHANNEL_COUNT’ festgelegt. Alle kanalbezogenen Signale (wie Zählerstände, Statusflags, Steuerbits) werden als Arrays der Länge ‘CHANNEL_COUNT’ deklariert. Dadurch werden nur so viele Flipflops und Logikelemente instanziiert, wie für die aktivierten Kanäle tatsächlich benötigt werden.

Nicht genutzte Kanäle belegen somit keine unnötigen Ressourcen, da die Generate-Blöcke und Prozesse ausschließlich für die Indizes ‘0’ bis ‘CHANNEL_COUNT-1’ erzeugt werden. Auch die Kanalzuordnung erfolgt über Mapping-Arrays, sodass keine redundante Logik für ungenutzte Ein-/Ausgänge entsteht.

Ein Beispiel aus dem Code:

```

1 type t_enc_count_array is array (0 to CHANNEL_COUNT-1) of std_logic_vector
2 signal enc_count, upper_limit, next_upper_limit : t_enc_count_array := (
3 ...
4 channel_gen : for i in 0 to CHANNEL_COUNT-1 generate
5     ...
6 end generate;

```

Auch die Steuer- und Feedbackschnittstellen werden als Bitfelder ('std_logic_vector') organisiert, sodass mehrere Steuer- und Statusinformationen platzsparend in einem einzigen Datenwort übertragen werden können. Dies reduziert den Bedarf an separaten Signalen und vereinfacht die Schnittstellenanbindung.

Im Fehlerfall oder bei CPU_STOP werden alle Status- und Ausgangssignale gezielt zurückgesetzt, sodass keine unnötigen Register oder Speicherbereiche dauerhaft belegt bleiben:

```

1 if CPU_STOP = '1' and WR_REC_Control = x"00000000" then
2     upper_Limit(i)      <= (others => '0');
3     next_upper_limit(i) <= (others => '0');
4     load_delay_cnt(i)   <= 0;
5     LimitReached(i)     <= '0';
6 end if;

```

Durch diese konsequente Nutzung von Arrays, Bitfeldern und Generate-Strukturen wird der FPGA optimal ausgelastet und der Ressourcenverbrauch auf das tatsächlich benötigte Minimum reduziert.

4.5 Fehler- und Diagnosestrategien

Das VHDL-Design implementiert eine systematische Fehlererkennung und sichere Fehlerbehandlung auf Kanalebene. Für jeden Kanal wird das Signal 'system_fault(i)' berechnet, das verschiedene Fehlerquellen überwacht:

- ****DI_QI_BAD****: Fehler am zugeordneten Digitaleingang (z.B. Drahtbruch, Kurzschluss) - ****DQ_QI_BAD****: Fehler am zugeordneten Digitalausgang - ****RS485_QI_BAD****: Fehler an den RS485-Ein-/Ausgängen (z.B. Kommunikationsfehler, Pegelstörung)
- ****LP_QI_BAD****: Fehler in der Spannungsversorgung (Low Power)

Die Fehler werden im Prozess 'fault_dq_proc' für jeden Kanal individuell ausgewertet. Sobald ein Fehler erkannt wird ('system_fault(i) = '1'), wird der entsprechende Ausgang (DQ oder RS485) automatisch in einen sicheren Zustand (SAFE State) überführt. Der SAFE State ist abhängig von der konfigurierten Polarität und sorgt dafür, dass im Fehlerfall keine unkontrollierten Aktionen ausgelöst werden.

Die Fehlerdetails werden über das Feedback-Interface (z.B. 'FB_IF7') an die SPS rückgemeldet. Hierbei werden die Fehlerbits für DI, DQ, RS485 und die Kanalnummer gesetzt, sodass die Steuerung gezielt auf den Fehler reagieren kann. Ist kein Fehler vorhanden, werden die Fehlerbits explizit gelöscht.

Zusätzlich wird im Fehlerfall der Ausgang sofort deaktiviert und der Status 'StartDQ_active' zurückgesetzt, um einen sicheren Anlagenzustand zu gewährleisten. Im Fall eines globalen CPU-Stop (z.B. durch 'CPU_STOP = '1'' und 'WR_REC_Control = x"00000000,,), werden alle Ausgänge ebenfalls in den SAFE State gesetzt und alle Statusregister zurückgesetzt.

Dieses Konzept stellt sicher, dass sowohl Einzelkanalfehler als auch globale Fehler jederzeit erkannt und sicher behandelt werden. Die SPS erhält über die Rückmeldebites eine detaillierte Diagnose und kann gezielt Maßnahmen einleiten.

Synchronisation und Zustandsautomaten

Die Synchronisation der Ladevorgänge und die Überwachung der Zählergrenzen erfolgt im VHDL-Design über einen einfachen, aber effektiven Zustandsautomaten pro Kanal, der durch das Signal `load_delay_cnt(i)` realisiert wird. Dieser Zustandsautomat steuert, wann ein neuer Sollwert (Obergrenze) aus dem azyklisch übertragenen `WR_REC` übernommen wird und wie auf synchrone Steuerbefehle wie `Load` oder `StartDQ` reagiert wird.

Der Ablauf ist wie folgt:

- Bei einer Flanke auf `WR_REC_NEW` wird der neue Sollwert für den jeweiligen Kanal in `next_upper_limit(i)` zwischengespeichert.
- Erst wenn das synchrone Steuersignal `Load(i)` gesetzt wird, übernimmt der Zustandsautomat im Zustand `load_delay_cnt(i) = 0` den neuen Wert in `upper_limit(i)` und prüft sofort, ob der aktuelle Zählerstand bereits das Limit erreicht hat.
- Der Automat wechselt in den Zustand 1, solange `Load(i)` aktiv ist, und kehrt nach Loslassen des Signals wieder in den Grundzustand zurück.
- Das Limit-Flag `LimitReached(i)` wird gesetzt, sobald der Zählerstand die Obergrenze erreicht oder ein Überlauf erkannt wird. Mit dem Steuersignal `StartDQ(i)` kann dieses Flag zurückgesetzt und ein neuer Dosierzyklus gestartet werden.

Durch diese Trennung von asynchroner Sollwertübertragung (über `WR_REC`) und synchroner Steuerung (über `CTRL_IF/Load`) wird sichergestellt, dass Werteänderungen nur gezielt und deterministisch übernommen werden. Gleichzeitig verhindert der Zustandsautomat Race Conditions und garantiert, dass keine Sollwertänderung während eines laufenden Zyklus unbemerkt bleibt.

Ein Auszug aus dem VHDL-Code illustriert dieses Prinzip:

```

1  process(CLK, RST)
2  begin
3      if RST = '1' then
4          upper_Limit(i)      <= (others => '0');
5          next_upper_limit(i) <= (others => '0');
6          load_delay_cnt(i)   <= 0;
7          LimitReached(i)    <= '0';
8      elsif rising_edge(CLK) then
9          if WR_REC_NEW = '1' then
10             next_upper_limit(i) <= std_logic_vector(unsigned(WR_RECxx) -
11         end if;
12         case load_delay_cnt(i) is
13             when 0 =>
14                 if Load(i) = '1' then
15                     upper_Limit(i)      <= next_upper_limit(i);
16                     load_delay_cnt(i) <= 1;
17                     if unsigned(enc_count(i)) >= unsigned(next_upper_li
18                         LimitReached(i) <= '1';
19                     else
20                         LimitReached(i) <= '0';
21                     end if;
22                 end if;
23             when 1 =>
24                 if Load(i) = '0' then
25                     load_delay_cnt(i) <= 0;
26                 end if;
27             when others =>
28                 load_delay_cnt(i) <= 0;
29         end case;
30         if StartDQ(i) = '1' then

```

```
31         LimitReached(i) <= '0';
32     end if;
33     if overflow(i) = '1' then
34         LimitReached(i) <= '1';
35     end if;
36 end if;
37 end process;
```

Dieses Konzept sorgt für eine robuste, deterministische und synchronisierte Steuerung aller Zähl- und Dosierprozesse im FPGA.

4.6 Erweiterbarkeit für zukünftige Anforderungen

Das vorgestellte VHDL-Design ist gezielt auf Erweiterbarkeit ausgelegt, um zukünftige Anforderungen wie zusätzliche Messmodi, neue Schnittstellen oder komplexere Diagnosemechanismen einfach integrieren zu können. Dies wird durch die konsequente Nutzung von Generics, Array-Typen und Generate-Blöcken erreicht. Die zentrale Konstante 'CHANNEL_COUNT' legt die Anzahl der Kanäle fest, sodass durch einfaches Erhöhen dieses Werts und Anpassen der Mapping-Arrays ('CHANNEL_MAP_IN_CH', 'CHANNEL_MAP_OUT_CH') weitere Kanäle hinzugefügt werden können, ohne die bestehende Logik zu verändern.

Neue Funktionen wie alternative Messverfahren (z.B. Frequenzmessung, Impulsbreitenmessung) lassen sich modular als zusätzliche Komponenten (wie 'FB82_Freq32_e') pro Kanal einbinden. Die Steuer- und Feedbackschnittstellen sind als Bitfelder und Arrays organisiert, sodass weitere Steuerbits oder Statusinformationen problemlos ergänzt werden können. Auch die Fehler- und Diagnoselogik ist pro Kanal gekapselt und kann durch zusätzliche Überwachungsmechanismen erweitert werden.

Durch die klare Trennung von Steuer-, Daten- und Diagnosepfaden sowie die Nutzung von Generate-Strukturen ist das Design flexibel anpassbar und für zukünftige Erweiterungen vorbereitet. Dies ermöglicht eine nachhaltige Weiterentwicklung der Applikation, ohne dass grundlegende Architekturänderungen erforderlich sind.

Ein Auszug aus dem VHDL-Code zeigt die zentrale Rolle der Generics und Arrays für die Erweiterbarkeit:

```

1 constant CHANNEL_COUNT : natural := 8;
2 type t_enc_count_array is array (0 to CHANNEL_COUNT-1) of std_logic_vector
3 signal enc_count, upper_Limit, next_upper_limit : t_enc_count_array := (
4 ...
5 channel_gen : for i in 0 to CHANNEL_COUNT-1 generate
6 end generate;

```

4.7 Testbarkeit und Simulation

Die Architektur des VHDL-Designs ist gezielt auf eine hohe Testbarkeit und einfache Simulation ausgelegt. Durch die konsequente Trennung von Steuer- und Datenpfaden sowie die Verwendung von Arrays und Generate-Blöcken kann jeder Kanal unabhängig simuliert und getestet werden. Die zentrale Steuerung erfolgt über klar definierte Schnittstellen (CTRL_IF, WR_REC), sodass gezielte Stimuli für einzelne Kanäle oder Funktionen einfach im Testbench gesetzt werden können.

Im gezeigten Code werden alle kanalbezogenen Signale als Arrays deklariert, z.B.:

```

1 type t_enc_count_array is array (0 to CHANNEL_COUNT-1) of std_logic_vector
2 signal enc_count, upper_Limit, next_upper_limit : t_enc_count_array := (

```

Dadurch lassen sich im Test gezielt einzelne Kanäle ansprechen, ohne Seiteneffekte auf andere Kanäle zu verursachen.

Die Steuer- und Feedbackschnittstellen sind ebenfalls klar voneinander getrennt und als Bitfelder organisiert. Dies ermöglicht es, in der Simulation gezielt Steuerbits (z.B. Load, StartDQ, CounterReset) für einzelne Kanäle zu setzen und die Reaktion des Designs zu beobachten.

Die Verwendung von Generate-Blöcken wie

```

1 channel_gen : for i in 0 to CHANNEL_COUNT-1 generate
2     — Kanal-Logik
3 end generate;

```

erlaubt es, die Logik für jeden Kanal separat zu simulieren und zu validieren. Fehlerfälle, Grenzwerte oder spezielle Betriebsmodi können so gezielt für einzelne Kanäle getestet werden.

Auch die Fehler- und Diagnoselogik ist pro Kanal gekapselt, sodass in der Simulation gezielt Fehlerbedingungen (z.B. DI_QI_BAD, DQ_QI_BAD) für einzelne Kanäle gesetzt und die Reaktion des Systems überprüft werden kann.

Durch diese modulare und strukturierte Architektur wird die Erstellung von Testbenches vereinfacht, die Testabdeckung erhöht und die Fehlersuche im Entwicklungsprozess deutlich beschleunigt.

4.8 Zyklische und azyklische Kommunikation

Das VHDL-Design trennt klar zwischen zyklischer und azyklischer Kommunikation, um Prozesssicherheit und Flexibilität zu maximieren. Die gesamte Kommunikation erfolgt über drei Schnittstellen: CTRL_IF (zyklische Steuerung), FB_IF (zyklisches Feedback), WR_REC (azyklische Sollwertübertragung) und RD_REC (azyklisches Auslesen).

Die zyklischen Schnittstellen sind für zeitkritische Steuerungsaufgaben konzipiert und haben jeweils eine Datenmenge von 8 Dwords. Bei einer Kanalanzahl von 14 bleibt dadurch nur 1 Word pro Kanal und 1 Dword für zusätzliche Parameter übrig. Dies ist ausreichend für die Übertragung von Steuer- und Statusinformationen, jedoch nicht für umfangreiche Sollwertänderungen.

Dafür ist die azyklische Kommunikation geeignet, da Sollwerte nur selten im laufenden Prozess geändert werden sollen und die azyklische Kommunikation eine einstellbare Datenmenge hat. Sie wurde auf 64 Byte, also 16 Dwords, eingestellt. Dadurch ist es möglich, alle Sollwerte der 14 Kanäle in einem einzigen azyklischen Transfer zu übertragen und noch 2 Dwords für zukünftige Erweiterungen oder zusätzliche Parameter zu reservieren.

Zyklische Steuerung (CTRL_IF): Zyklische Steuerbefehle werden über die CTRL_IF-Schnittstelle übertragen und steuern in jedem SPS-Zyklus zentrale Funktionen wie Reset, Load, Hold, StartDQ oder EnableDQ für jeden Kanal. Diese Steuerbits werden im Prozess `ctrl_fb_mapping_proc` aus den CTRL_IF-Worten

extrahiert und auf die internen Steuersignale der jeweiligen Kanäle gemappt. Dadurch ist eine deterministische, taktsynchrone Steuerung aller Kanäle gewährleistet.

Zyklisches Feedback (FB_IF): Das Feedback-Interface (FB_IF) liefert in jedem SPS-Zyklus aktuelle Status- und Prozesswerte für alle Kanäle. Jeder Kanal stellt ein 16-Bit-Wort bereit, dessen Bitstruktur wie folgt aufgebaut ist:

- **Bit 0–14:** Aktueller Prozesswert (z.B. Zählerstand oder Frequenz).
- **Bit 15:** Spiegelung des `Select_Feedback_Value`-Flags (Diagnose/Rückmeldung).
- **Bit 16 (MSB):** Status des Ventilausgangs (1 = Ausgang ein, 0 = Ausgang aus).

Zusätzlich werden über FB_IF Fehler- und Statussignale übertragen:

- **ErrorChannelNumber:** Nummer des letzten Kanals mit Fehler (0 = kein Fehler).
- **DI_DQ_BAD:** Bit-codierter Fehlerstatus für Digitaleingänge/-ausgänge.
- **RS485_LP_BAD:** Bit-codierter Fehlerstatus für RS485 oder L+.
- **StatusCode:** Applikations-Fehlercode und Aktivitätsanzeige.

Damit ist eine kontinuierliche Überwachung und Diagnose aller Kanäle durch die SPS möglich.

Azyklische Sollwertübertragung (WR_REC): Azyklische Sollwertübertragungen erfolgen über die WR_REC-Schnittstelle. Neue Obergrenzen (Sollwerte) für die Zählerkanäle werden als 32-Bit-Werte übertragen und im Prozesszyklus bei einer Flanke auf `WR_REC_NEW` in die Buffer-Variable `next_upper_limit(i)` übernommen. Erst wenn das zyklische Steuersignal `Load(i)` gesetzt wird, übernimmt der Kanal den neuen Sollwert in die aktive Obergrenze (`upper_Limit(i)`). Dadurch wird verhindert, dass Sollwertänderungen während eines laufenden Dosierzyklus unbemerkt übernommen werden.

Dieses Zusammenspiel sorgt für hohe Prozesssicherheit: Azyklische Änderungen werden gepuffert und erst durch ein gezieltes, zyklisches Steuersignal wirksam. Gleichzeitig bleibt das System flexibel, da Sollwerte jederzeit aktualisiert werden

können, ohne den laufenden Betrieb zu stören. Die klare Trennung der Kommunikationswege und die Synchronisation über Steuersignale verhindern Race Conditions und ermöglichen eine deterministische, fehlertolerante Steuerung aller Kanäle.

Ein Auszug aus dem VHDL-Code illustriert das Prinzip:

```

1  if WR_REC_NEW = '1' then
2      next_upper_limit(i) <= std_logic_vector(unsigned(WR_RECxx) - 1);
3  end if;
4  ...
5  if Load(i) = '1' then
6      upper_Limit(i) <= next_upper_limit(i);
7  end if;

```

4.9 Sicherheit bei CPU_STOP

Im Fehlerfall oder bei einem CPU_STOP muss das System in einen definierten sicheren Zustand übergehen, um unkontrollierte Aktionen und potenzielle Gefährdungen zu verhindern. Dies wird im VHDL-Design durch gezielte Abfragen und Rücksetzungen aller relevanten Steuer- und Ausgangssignale realisiert.

Im gezeigten Code wird der sichere Zustand insbesondere durch die Abfrage von CPU_STOP und WR_REC_Control = x"00000000" in mehreren Prozessen sichergestellt:

- **ctrl_fb_mapping_proc:** Sobald ein CPU_STOP erkannt wird und das Kontrollwort auf Null steht, werden für alle Kanäle die Steuerbits wie **EnabledQ**, **Load**, **HoldSW**, **StartDQ** und **CounterReset** explizit auf '0' bzw. '1' (Reset) gesetzt. Damit werden alle Ausgänge deaktiviert und alle Statusregister zurückgesetzt.
- **channel_gen:** Im FSM-Prozess für jeden Kanal werden im CPU_STOP-Fall die Obergrenzen, Puffer und Statusflags wie **upper_Limit**, **next_upper_limit**, **load_delay_cnt** und **LimitReached** auf Null bzw. Grundzustand zurückgesetzt.

- **fault_dq_proc:** Die Ausgangssignale (DQ) und der Status StartDQ_active werden im Fehlerfall oder bei CPU_STOP explizit auf SAFE-State ('0') gesetzt. Damit wird sichergestellt, dass keine Ausgänge mehr aktiv sind.

Ein Auszug aus dem Code illustriert das Verhalten:

```

1   DQ(i)          <= '0';
2   StartDQ_active <= (others => '0');
3   upper_Limit(i) <= (others => '0');
4   next_upper_limit(i) <= (others => '0');
5   load_delay_cnt(i) <= 0;
6   LimitReached(i) <= '0';
7 end if;
```

Durch diese Maßnahmen wird garantiert, dass das System im Fehlerfall oder bei einem CPU_STOP deterministisch und zuverlässig in einen sicheren Zustand übergeht. Dies entspricht den Anforderungen an funktionale Sicherheit in industriellen Anwendungen und verhindert ungewollte Schalthandlungen oder undefinierte Zustände.

4.10 Konzept und Architektur des neuen Applikationsdesigns

Byte Zuweisung für FB_IF (32 Byte)

- 3 Byte für Kanalobergrenze = 4095 (Standardventil ist 1 Impuls/ 1 ml -> 4l abfüllung) => bis zu 10 Kanäle möglich
- 4 Byte für Kanalobergrenze = 65535 (65l abfüllung) => nur 8 Kanäle möglich

Andere Ideen: Vielleicht auf TMFASTUserWriteREC ausweichen? (Bis zu 128 Byte) asynchron (was ist wenn daten mitten im ablauf geändert werden?)

Algorithmus, der im ersten Zyklus die Kanalobergrenzen 1–4 von der CPU an das TMFast sendet, im zweiten Zyklus 5–8 usw. ist synchron und eignet sich besser

die Steuerschnittstelle, da über die im Betrieb auch taktsynchron neue Werte geschrieben werden können. Der Nachteil des Write-Records ist die Asynchronität zum Zyklus.

Zähler -> Unter- und Obergrenze einstellbar (nur Obergrenze)

CPU Stop beachten:

Das umgesetzte Konzept besteht daraus, die Obergrenze aller geladenden Kanäle (im bezug auf "dynamisches" laden der Kanalmänge -> wichtig, da sonst 1400 Logikelemente (16 DWord für 12 Kanäle) zur speicherung nutzlos verbraucht werden) azyklisch über den TMFASTUserWriteREC zu verschicken. Bietet die Möglichkeit 1 bis 32 DWord (4 - 128 Byte) an länge zu verschicken. Ein DWord (ca. 130 Logikelemente) reicht also für einen Kanal um eine 32 Bit zahl zu speichern, die die Obergrenze des Kanals angibt. Eine 32 Bit Zahl ohne Vorzeichen (4294967295) ist für die meisten Anwendungen ausreichend, da sie 4.294.967.295 Impulse (z.B. 1 Impuls = 1 ml) darstellen kann, was 4294967295 ml oder 4294,967295 l entspricht. Die Steuerungswerte wie z.B Reset, Load oder Hold werden über die zyklischen Schnittstellen CTRL_IF über 1 DWord große Bitmuster übertragen und die Statuswerte über die zyklischen Schnittstellen FB_IF zurückgegeben.

4.11 Überlaufkompensation (Dynamic Overflow Compensation)

Diese Methode wird für präzise Dosierungssysteme verwendet, bei denen der Nachlaufeffekt nach Ventilschließung berücksichtigt werden muss.

4.11.1 Vorteile

- Berücksichtigung der Systemträgheit (mechanische Nachlaufzeit des Ventils)
- Kompensation des systematischen Fehlers durch den Nachlauf
- Bei konstanten Bedingungen (Druck, Temperatur, Viskosität) sehr genau
- Keine zusätzliche Hardware erforderlich

- Selbstlernend/adaptiv, wenn der letzte Überlauf für die nächste Dosierung verwendet wird
- Gut für kontinuierliche Prozesse geeignet

4.11.2 Nachteile

- Bei schwankenden Bedingungen (Druck, Temperatur, Viskosität) kann der Überlauf variieren
- Die erste Dosierung nach einem Neustart hat keine Überlaufkompensation
- Bei Störungen (z.B. Luftblasen) wird ein falscher Überlaufwert für die nächste Dosierung verwendet
- Bei Produktwechsel muss das System neu einlernen
- Bei sehr unterschiedlichen Füllmengen muss eventuell mit verschiedenen Überlaufwerten gearbeitet werden

4.11.3 Alternative Methoden

1. Prädiktion des Abschaltzeitpunkts

- Berechnung basierend auf aktueller Durchflussrate
- Berücksichtigung der Ventil-Schließzeit

2. Zweistufige Dosierung

- Schnelle Befüllung (ca. 90%)
- Langsame Präzisionsbefüllung

3. PID-geregelte Dosierung

- Kontinuierliche Regelung
- Hohe Präzision

4. Massendurchflussmesser

- Direkte Massenmessung

- Höchste Genauigkeit

4.11.4 Mathematische Beschreibung

Für die Überlaufkompensation gilt:

$$V_{target} = V_{desired} - V_{overflow_previous} \quad (4.1)$$

Wobei:

V_{target} = Ziel-Abschaltvolumen

$V_{desired}$ = Gewünschtes Endvolumen

$V_{overflow_previous}$ = Gemessener Überlauf der vorherigen Dosierung

4.12 Analyse der Überlaufkompensation

Eine zusätzliche Berechnung der Überlaufkompensation ist nicht vorteilhaft, da das System bereits eine optimale Implementierung aufweist.

4.12.1 Aktuelle Implementierung (automatisch)

- **Kontinuierlicher Zählvorgang:**

$$Count_{total} = Count_{target} + Count_{overflow} \quad (4.2)$$

- **Ventilsteuerung:**

$$Valve_{state} = \begin{cases} 0 & \text{wenn } Count_{current} \geq Count_{target} \\ 1 & \text{wenn } Count_{current} < Count_{target} \end{cases} \quad (4.3)$$

- **Automatische Nachlaufkompensation:**

– Nachlaufmenge wird direkt zum nächsten Zyklus addiert

- Keine Zwischenspeicherung notwendig
- Keine Berechnungszeit erforderlich

4.12.2 Vorteile der automatischen Implementierung

1. Effizienz:

- Keine zusätzlichen Berechnungsschritte
- Keine Verzögerungen im Prozessablauf
- Optimale Ressourcennutzung

2. Genauigkeit:

- Keine Rundungsfehler durch Berechnungen
- Direkte Messung statt berechneter Werte
- Kontinuierliche Erfassung ohne Unterbrechung

3. Zuverlässigkeit:

- Reduzierte Komplexität
- Weniger potenzielle Fehlerquellen
- Robustes Systemverhalten

4.12.3 Nachteile einer zusätzlichen Berechnung

- **Erhöhte Komplexität:**
 - Zusätzliche Prozesse und Variablen
 - Komplexere Wartung und Debugging
 - Schwierigere Systemanalyse
- **Potenzielle Fehlerquellen:**
 - Überläufe bei arithmetischen Operationen

- Timing-Probleme bei Berechnungen
- Synchronisationsfehler zwischen Prozessen
- **Ressourcenverbrauch:**
 - Zusätzlicher Speicherbedarf
 - Erhöhte CPU-Last
 - Mehr FPGA-Ressourcen benötigt
- **Keine Genauigkeitsverbesserung:**
 - Keine Verbesserung der Messgenauigkeit
 - Mögliche Verschlechterung durch Rundungsfehler
 - Unnötige Komplexität ohne Mehrwert

4.12.4 Mathematische Betrachtung

Die Gesamtmenge einer Dosierung lässt sich beschreiben als:

$$V_{total} = V_{target} + V_{overflow} \quad (4.4)$$

Wobei im automatischen System gilt:

$$V_{next_target} = V_{desired} - V_{overflow_previous} \quad (4.5)$$

Dies wird implizit durch den kontinuierlich laufenden Zähler realisiert, ohne explizite Berechnung.

Warum die Kanalbelegung? RS485 Eingänge sind Strong deshalb mehr

4.13 Verteilung von RS485- und DIQ-Kanälen: Flexibilität und Varianten

Die Verteilung der Kanäle auf RS485, DIQ (Digital Input/Output) und reine Digitaleingänge ist ein zentrales Merkmal des Designs und beeinflusst maßgeblich die Flexibilität und Anpassbarkeit des Systems an verschiedene Applikationen. Im gezeigten VHDL-Code wird dies durch die Mapping-Arrays `CHANNEL_MAP_IN_CH` und `CHANNEL_MAP_OUT_CH` realisiert, die für jeden logischen Kanal festlegen, welcher physikalische Eingang bzw. Ausgang verwendet wird.

Standardmäßig werden die ersten acht Kanäle als klassische DI/DQ-Kanäle betrieben, während die weiteren Kanäle flexibel als RS485 oder zusätzliche DIQ-Kanäle genutzt werden können. Die Auswahl des Eingangstyps erfolgt dynamisch im Code:

```
1 input_selector(i) <= RS485_RX(CHANNEL_MAP_IN_CH(i)) when i > 8 else DI(Q
```

Dadurch kann für jeden Kanal individuell entschieden werden, ob ein 24V-Digitaleingang (DI) oder ein RS485-Pegel verwendet wird. Die Ausgänge werden analog über das Mapping auf DQ oder RS485_TX verteilt.

Variante: Maximale Anzahl an 24V-Digitaleingängen

Ein besonderer Vorteil des Designs ist die Möglichkeit, alle verfügbaren DIQ-Kanäle als reine Digitaleingänge zu betreiben. Dazu werden die Mapping-Arrays so angepasst, dass alle Kanäle auf DI-Eingänge zeigen und keine RS485-Kanäle verwendet werden. Dies ist insbesondere dann sinnvoll, wenn eine Applikation eine große Anzahl an 24V-Eingängen benötigt, beispielsweise für Zähl- oder Überwachungsaufgaben. Die Hardware-Ressourcen werden so optimal für die jeweilige Anwendung genutzt.

Variante: Gemischte Nutzung und individuelle Anpassung

Alternativ können die Kanäle beliebig zwischen DI, DQ und RS485 aufgeteilt werden, indem die Mapping-Arrays entsprechend konfiguriert werden. So lassen sich beispielsweise vier Kanäle als RS485 (für schnelle serielle Kommunikation), vier als DIQ (für klassische Ein-/Ausgänge) und weitere als reine DI nutzen. Die Anpassung erfolgt ausschließlich über die Mapping-Arrays, ohne dass die Kanal- oder Prozesslogik geändert werden muss.

Vorteile der flexiblen Kanalverteilung

- **Anpassbarkeit:** Die Kanalbelegung kann projektspezifisch auf die benötigten Signaltypen (DI, DQ, RS485) abgestimmt werden.
- **Skalierbarkeit:** Durch einfaches Ändern der Mapping-Arrays kann die Anzahl der Eingänge/Ausgänge je nach Bedarf maximiert werden.
- **Wiederverwendbarkeit:** Das gleiche VHDL-Design kann für unterschiedliche Hardware-Konfigurationen genutzt werden.
- **Ressourceneffizienz:** Nicht benötigte Funktionen (z.B. RS485) belegen keine FPGA-Ressourcen, wenn sie nicht gemappt werden.

Nachteile und Grenzen

- **Hardwareabhängigkeit:** Die tatsächliche Verfügbarkeit von DI, DQ und RS485-Kanälen hängt von der Hardwareausstattung ab.
- **Komplexität der Konfiguration:** Eine fehlerhafte Zuordnung in den Mapping-Arrays kann zu unerwartetem Verhalten führen.
- **Signalqualität:** RS485-Kanäle bieten andere elektrische Eigenschaften als klassische 24V-DI, was bei der Applikationsplanung zu berücksichtigen ist.

Fazit: Die flexible Verteilung der Kanäle auf RS485, DIQ und DI ermöglicht eine optimale Anpassung an verschiedenste Anforderungen – von maximaler Eingangs-dichte bis hin zu gemischten Kommunikationsszenarien. Die Konfiguration erfolgt zentral und ohne Anpassung der Prozesslogik, was Wartung und Erweiterung erheblich vereinfacht.

(Implementierung des Frequenzmessers / Vergleich von FM350-2 (ist im Handbuch gut erklärt) und TMFAST)

4.14 Umsetzung der Funktionalität in VHDL

-Diagramme einfügen, beschreiben und analysieren

4.15 Test und Validierung der VHDL-Implementierung

-Questa simuliert Eingänge und prüft die Ausgänge

4.16 Schnittstelle mit dem TIA Portal

-Diagramm für Schnittstelle und Funktion der TIA Anwendung

4.17 Vergleich mit der bisherigen Lösung

-Auf abschnitte in der Analyse eingehen

////////////////////////////////////
////////////////////////////////////
////////////////////////////////////

5 Entwicklung des VHDL-basierten Applikationsbeispiels

5.1 Anforderungen und Architektur

Für die Neuentwicklung des Zählermoduls auf Basis des TM FAST ergeben sich verschiedene funktionale und technische Anforderungen: Das Modul soll mindestens acht unabhängige Zählerkanäle bereitstellen, die flexibel als Digitaleingänge, Digitalausgänge oder RS485/TTL-Kanäle genutzt werden können. Jeder Kanal arbeitet mit einer ausreichend breiten Zählvariable (typischerweise 32 Bit), um große Wertebereiche abzudecken. Die Übergabe von Sollwerten und Parametern muss effizient und im laufenden Betrieb möglich sein, ohne den Prozess zu unterbrechen. Betriebsmodi wie Rundenzähler, Reset, Hold und Start/Stop-Funktionen sind zu unterstützen. Statusinformationen und Prozesswerte (z.B. aktueller Zählerstand, Geschwindigkeit) werden für jeden Kanal rückgemeldet. Fehler müssen erkannt, sicher behandelt und an die Steuerung gemeldet werden. Die Lösung ist vollständig mit dem TIA Portal integrierbar und zur FM350-2 kompatibel. Ressourcenschonende Implementierung und einfache Erweiterbarkeit sind zentrale Ziele.

5.2 Modularisierung und Flexibilität

Das VHDL-Design ist konsequent modular aufgebaut. Die Anzahl der Kanäle wird zentral über das Generic `CHANNEL_COUNT` festgelegt. Kanalbezogene Signale und Zustände werden als Arrays deklariert, sodass die Logik für jeden Kanal identisch und unabhängig instanziiert werden kann. Generate-Blöcke erzeugen die komplette Zähler- und Frequenzmesslogik pro Kanal automatisch. Neue Kanäle können durch Erhöhen von `CHANNEL_COUNT` und Anpassen der Mapping-Arrays hinzugefügt werden, ohne die Prozesslogik zu verändern.

Die Kanaluordnung erfolgt über Mapping-Arrays (`CHANNEL_MAP_IN_CH`, `CHANNEL_MAP_OUT_CH`), die für jeden logischen Kanal den verwendeten physikalischen Eingang bzw. Ausgang festlegen. Dadurch kann das Design einfach an unterschiedliche Hardware-Konfigurationen oder Kundenanforderungen angepasst werden, ohne die Kanal-Logik zu ändern.

5.3 Ressourcenoptimierung

Arrays und Bitfelder minimieren gezielt den Ressourcenverbrauch im FPGA. Nur die tatsächlich verwendeten Kanäle werden instanziiert, nicht genutzte Kanäle belegen keine Ressourcen. Steuer- und Feedbackschnittstellen sind als Bitfelder organisiert, sodass mehrere Informationen platzsparend in einem Datenwort übertragen werden. Im Fehlerfall oder bei `CPU_STOP` werden alle Status- und Ausgangssignale gezielt zurückgesetzt, sodass keine unnötigen Register oder Speicherbereiche dauerhaft belegt bleiben.

5.4 Fehler- und Diagnosestrategien

Jeder Kanal verfügt über eine eigene Fehlererkennung (`system_fault(i)`), die verschiedene Fehlerquellen überwacht (z.B. `DI_QI_BAD`, `DQ_QI_BAD`, `RS485_QI_BAD`, `LP_QI_BAD`). Im Fehlerfall wird der Ausgang automatisch in einen sicheren Zustand (SAFE State) überführt. Fehlerdetails werden über das Feedback-Interface an die SPS rückgemeldet. Bei globalem CPU-Stop werden alle Ausgänge ebenfalls in den SAFE State gesetzt und Statusregister zurückgesetzt.

5.5 Synchronisation und Zustandsautomaten

Die Synchronisation der Ladevorgänge und die Überwachung der Zählergrenzen erfolgt über einen einfachen Zustandsautomaten pro Kanal. Asynchron übertragene Sollwerte werden gepuffert und erst durch ein synchrones Steuersignal (`Load`) übernommen. Das verhindert Race Conditions und garantiert, dass Sollwertänderungen nur gezielt und deterministisch übernommen werden.

5.6 Erweiterbarkeit und Testbarkeit

Durch die Nutzung von Generics, Array-Typen und Generate-Blöcken ist das Design einfach erweiterbar. Neue Kanäle oder Funktionen können modular ergänzt werden. Die klare Trennung von Steuer-, Daten- und Diagnosepfaden sowie die Nutzung von Arrays und Generate-Strukturen erleichtern die Simulation und gezielte Tests einzelner Kanäle.

5.7 Kommunikationskonzept: Zyklisch vs. azyklisch

Das Design trennt klar zwischen zyklischer (CTRL_IF) und azyklischer (WR_REC) Kommunikation. Zyklische Steuerbefehle steuern zentrale Funktionen wie Reset, Load, Hold, StartDQ oder EnableDQ. Azyklische Sollwertübertragungen werden gepuffert und erst durch ein zyklisches Steuersignal wirksam. Dadurch bleibt das System flexibel und prozesssicher. Jeder Kanal stellt ein 16-Bit-Wort bereit, das Prozesswert, Diagnose- und Statusinformationen enthält. Fehler- und Statussignale sind klar codiert und ermöglichen eine gezielte Diagnose durch die SPS.

5.8 Sicherheit bei CPU_STOP

Im Fehlerfall oder bei CPU_STOP geht das System in einen sicheren Zustand über. Steuer- und Ausgangssignale werden zurückgesetzt, alle Ausgänge deaktiviert und Statusregister auf den Grundzustand gesetzt. Dies entspricht den Anforderungen an funktionale Sicherheit in industriellen Anwendungen.

5.9 Überlaufkompensation

Die Überlaufkompensation berücksichtigt den Nachlauffeffekt nach Ventilschließung. Im aktuellen Design wird der Überlauf automatisch zum nächsten Zyklus addiert, ohne zusätzliche Berechnungsschritte. Das System ist effizient, genau und zuverlässig, da keine expliziten Berechnungen oder Zwischenspeicherungen notwendig sind.

warum geeignet

5.10 Flexible Kanalverteilung

Die Verteilung der Kanäle auf RS485, DIQ und DI erfolgt flexibel über die Mapping-Arrays. So kann das Design projektspezifisch auf die benötigten Signaltypen abgestimmt werden. Die Anpassung erfolgt zentral, ohne Änderungen an der Prozesslogik.

warum die verteilung auf RS485, DIQ und DI nur a geber

5.11 Vergleich mit bisherigen Lösungen

Im Vergleich zur FM350-2 bietet das neue Design eine höhere Flexibilität, bessere Ressourcenausnutzung und einfachere Erweiterbarkeit. Die klare Trennung von Steuer-, Daten- und Diagnosepfaden sowie die flexible Kanalzuordnung ermöglichen eine optimale Anpassung an unterschiedliche Applikationen.

5.12 Test und Validierung

Die modulare Architektur erleichtert die Simulation und Validierung. Jeder Kanal kann unabhängig getestet werden, Fehlerfälle und Grenzwerte lassen sich gezielt simulieren. Die klare Schnittstellendefinition unterstützt die Integration in das TIA Portal und die Anbindung an die SPS.

5.13 Fazit

Das vorgestellte VHDL-Design erfüllt die Anforderungen an Flexibilität, Erweiterbarkeit, Ressourceneffizienz und Sicherheit. Durch die konsequente Modularisierung und die klare Trennung der Kommunikationswege ist das System robust, wartbar und zukunftssicher.

6 Fazit und Ausblick

6.1 Zusammenfassung der Ergebnisse

6.2 Kritische Reflexion und Herausforderungen

6.3 Potenzielle Weiterentwicklungen und zukünftige Anwendungen

Literaturverzeichnis

- [1] Liam Bee (2022): *PLC and HMI development with Siemens TIA Portal: develop PLC and HMI programs using standard methods and structured approaches with TIA Portal V17*. Packt Publishing, Limited.
- [2] Siemens AG (2011): *Zählerbaugruppe FM 350-2 Gerätehandbuch*. Verfügbar unter: https://cache.industry.siemens.com/dl/files/178/1105178/att_20226/v1/s7300_fm350_2_operating_instructions_de_de-DE.pdf. Zugriff: 11. März 2025.
- [3] Siemens AG (2023): *TMFast Programmeier Handbuch*. Verfügbar unter: https://cache.industry.siemens.com/dl/files/088/109816088/att_1144842/v2/s71500_tm_fast_programming_manual_de-DE_de-DE.pdf. Zugriff: 12. März 2025.
- [4] IPH - Institut für Integrierte Produktion Hannover gGmbH: *Automatisierungspyramide: Definition, Bedeutung, Ebenen*. Verfügbar unter: <https://www.iph-hannover.de/de/dienstleistungen/automatisierungstechnik/automatisierungspyramide/>. Zugriff: 26. Juli 2025.
- [5] H. Berger: *Automatisieren mit SIMATIC S7-1500: Projektieren, Programmieren und Testen mit STEP 7 Professional*. 2. Aufl. Erlangen: Publics Publishing, 2017.
- [6] W. Babel: *Systemintegration in Industrie 4.0 und IoT: Vom Ethernet bis hin zum Internet und OPC UA*. Wiesbaden: Springer Fachmedien, 2024. DOI: <https://doi.org/10.1007/978-3-658-42987-4>.
- [7] Siemens AG: *Totally Integrated Automation Portal*. siemens.com. Verfügbar unter: <https://new.siemens.com/de/de/produkte/automatisierung/industrie-software/automatisierungs-software/tia-portal.html> (abgerufen am 26.07.2025).

- [8] Siemens AG: *SIMATIC Energy Suite*. siemens.com. Verfügbar unter: <https://www.siemens.com/de/de/produkte/automatisierung/industrie-software/automatisierungs-software/energiemanagement/simatic-energy-suite.html> (abgerufen am 26.07.2025).
- [9] Siemens AG: *SIMATIC ET 200MP*. siemens.com. Verfügbar unter: <https://new.siemens.com/de/de/produkte/automatisierung/systeme/industrie/io-systeme/simatic-et-200mp.html> (abgerufen am 26.07.2025).
- [10] Intel® Corp.: *Intel Cyclone 10 LP Device Overview*, 2022. [Online] Verfügbar unter: <https://www.intel.com/content/www/us/en/docs/programmable/683879/current/device-overview.html> (abgerufen am 27.07.2025).
- [11] G. Wild *et al.*: *Feature Specification for TM FAST*, Version 2.0, 2025.
- [12] Encoder Products Company: *The Basics of How an Encoder Works*, White Paper (WP-2011), rev. 15/08/25, 2025. [Online] Verfügbar unter: https://www.encoder.com/hubfs/white-papers/WP-2011_Basics/wp2011-basics-how-an-encoder-works.pdf (abgerufen am 15. August 2025).
- [13] Sensoray, Inc.: *Introduction to Incremental Encoders*, Application Note. [Online] Verfügbar unter: <https://www.sensoray.com/support/appnotes/encoders.htm> (abgerufen am 15. August 2025).
- [14]

7 Plan

Hier sind einige Schritte, die Ihnen dabei helfen können, eine VHDL-Applikation für das TM FAST-Modul zu entwickeln, um das alte FM350-2-Modul zu ersetzen:

Anforderungsanalyse

- Analysieren Sie sorgfältig die Funktionalität und Schnittstellen des FM350-2-Moduls.
- Identifizieren Sie die Kernfunktionen, die das neue TM FAST-Modul erfüllen muss.
- Definieren Sie klar die Eingangs- und Ausgangssignale, die Kommunikationsprotokolle und andere relevante Spezifikationen.

Architekturentwurf

- Entwerfen Sie eine modulare Architektur für Ihre VHDL-Applikation.
- Unterteilen Sie die Funktionalität in logische Blöcke wie Steuerungslogik, Kommunikationsschnittstellen, Datenpfade usw.
- Legen Sie die Schnittstellen zwischen den einzelnen Modulen fest.

VHDL-Entwicklung

- Beginnen Sie mit der Implementierung der einzelnen VHDL-Module basierend auf Ihrem Architekturentwurf.
- Achten Sie auf eine strukturierte und modulare Codeerstellung, um die Wartbarkeit und Erweiterbarkeit zu erleichtern.
- Verwenden Sie VHDL-Sprachkonstrukte wie Prozesse, Signale, Typen und Pakete, um Ihre Applikation übersichtlich und effizient zu gestalten.

Simulation und Test

- Erstellen Sie umfangreiche Testbänke, um Ihre VHDL-Applikation gründlich zu überprüfen.
- Führen Sie Simulationen durch, um das korrekte Verhalten der einzelnen Module und der Gesamtapplikation zu verifizieren.
- Testen Sie die Applikation schrittweise, um mögliche Fehler frühzeitig zu erkennen und zu beheben.

Integration und Validierung

- Integrieren Sie Ihre VHDL-Applikation in die Gesamtarchitektur des TM FAST-Moduls.
- Testen Sie die Applikation in der realen Umgebung, um ihre Kompatibilität und Funktionalität zu validieren.
- Passen Sie die Applikation gegebenenfalls an, um Anforderungen und Spezifikationen des TM FAST-Moduls zu erfüllen.

Dokumentation und Versionskontrolle

- Erstellen Sie eine ausführliche Dokumentation, die die Architektur, Schnittstellen, Konfiguration und Verwendung Ihrer VHDL-Applikation beschreibt.
- Nutzen Sie ein geeignetes Versionskontrollsystem, um Änderungen an Ihrer Applikation nachzuverfolgen und zu verwalten.

Durch diese strukturierte Vorgehensweise können Sie sicherstellen, dass Ihre VHDL-Applikation für das TM FAST-Modul die Anforderungen des FM350-2-Moduls erfüllt und in die Gesamtarchitektur des TM FAST-Systems nahtlos integriert wird.

8 Fragen

Müssen Alarme umgesetzt werden? -> Müssen nicht berücksichtigt werden
Müssen Hardwareconfigs von der CPU oder von TMFast gehandhabt werden? -> hardwareconfig
der (z.B. Ventile) ist in VHDL (andere Kommunikationsprotokolle und co pro z.B.
Ventile) Wie verhalten sich die In- und Outputs?

- Sollen alle Eingänge Zähl- und Richtungseingang sein oder Aufteilung zwischen den beiden? Nur zähler
-

wie funktionieren ventile überhaupt?

bit an die cpu senden

nachlauf bei inkrement zum abfüllen

cpu macht - tm fast macht ventil zu

auf ist 1 zu ist low für ventil

abfüllende menge muss übergeben werden

vhdl auf hoher ebene darstellen (als baustein)

synchron oder asynchron (für zähler oder generell) -> Alles Synchron (TM Fast arbeitet (einstellbar mit 50mHz) schneller als ein ventil) -> dadurch zählt es schneller als inkremente des ventils

clk wird bei änderung aktiviert (High->Low / Low->High)

steuerung des ventils über PLC oder TMFAST? Erstmal Plc mit bool senden (überlauf für counter (justierung)) / TMFast erfordert Kommunikations-Protokolle
Filter?

Anwendung in mehreren oder einem Process in vhdl?

soll der timer nur zählen, wenn die flanke positiv wird? oder auch wenn sie negativ wird? -> nur bei positiv

Allgemein: Schriftgröße und Seitenränder? Zitieren von Handbüchern (Graue Literatur) und ChatGPT? Mit Disclaimer

ES IST KEIN PRAXISBERICHT (KEIN PROZESS BESCHREIBEN) / EVTL: VORNACHTEILANALYSE

Vielleicht auf WR_REC ausweichen, wenn FB_IF nicht groß genug?

Muss der Siemens Betreuer einen Bachelor haben? Was kommt auf den Betreuer zu (Organisatorisch)

Steuerung eines Ventils von TMFAST? Über DQ -> Wie und was wird dafür benötigt, reicht es ein signal zu senden?

Ventile 1. Inkrementalgeber mit zwei Spuren (A/B-Signal): Viele Durchflussmesser oder Antriebssysteme (z.B. Drehgeber) geben zwei phasenversetzte Impulssignale aus: Spur A und Spur B.

Anhand der Phasenlage erkennt die CPU die Richtung:

Wenn A vor B kommt → Vorwärts

Wenn B vor A kommt → Rückwärts

Diese Technik nennt sich Quadraturauswertung. 24 V (HTL) sind Standardventil muss aber auch andere können:

nur die Anzahl der Impulse wichtig? 12 Kanäle sind nur 6 Alternativen? RS485 für Impulse -> JA? TTL-Signals Transistor-Transistor-Logik?

Willi: A/B oder A/B/N Geber? RS422? 24V für DI und 5V für CH. Anforderungen: Gleiche Technologie? flexibel (wie umsetzen)? Nur einen Eingang (A) war bei 350-2 mit (A/B) und 24V (DI) als Eingang

Maybe:

- A -> 5V (RS-485) * 8 / (DI) 24V * 8 / (DIQ) 24V * 4 / (DQ) 24V * 8
- A/B -> 5V * 4 / (DI) 24V * 4 / (DIQ)? 24V * 2
- A/B/N -> 5V * 2 / (DI+DIQ?) 24V * 3

+CH0 kann nicht beides verwendet werden -> EIA-485, auch als RS-485, benutzt ein Leitungspaar, um den invertierten und einen nichtinvertierten Pegel eines 1-Bit-Datensignals zu übertragen. Am Empfänger wird aus der Differenz der beiden Spannungspegel das ursprüngliche Datensignal rekonstruiert

wo finde ich noch informationen zum software reset (ResetSW)?

Wie funktionieren RS422/RS485 im vergelich zu DI in VHDL? Erweiterung auf mehrere Kanäle? In die vorhandenen oder als neue Prozesse? Wie Initialisiert man andere Kanäle dynamisch? Um Speicherplatz zu sparen. link in logikgruppe Gibt es einen Counter/Timer den man auf 1 Sekunde stellen kann? frequenzmesser/perioden messung

9 Wichtig

Effectuated Interfaces

Control interface: control interface, direction: from CPU to TM FAST

Feedback interface: feedback interface, direction: from TM FAST to CPU

Read record: asynchronous read record data from TM FAST to CPU

Write record: asynchronous write record data from CPU to TM FAST

Anforderungen: Vergleichswertübergabe azyklische über UserWriteREC 8x 24V Geber (evtl. auf 12 Upgraden mit DIQ) CounterReset -> Setze Counter=0 CounterHold -> Ignoriere Geber-Impulse ResetSW -> Setzt MAX_VALUE=0 und Counter=0 Load -> Setzt MAX_VALUE und Counter=0 (Muss, weil wenn MAX_VALUE > Counter kein Fehler (Der nutzer darf dafür nicht einbüßen))

- Es gibt 12 Kanäle. Ich würde jeweils die Eingänge und Ausgänge aus der gleichen Leistungsklasse verknüpfen (DI-DQ; Umschaltbare DIQ; RS485)
- Der Sollwert, an dem abgeschaltet werden soll, wird über den Anwenderdatensatz übermittelt, Datensatz mit 32 Byte, 2 Byte Kennung und dann 14 mal der Sollwert als Integer (2 byte). Oder direkt in der Steuerschnittstelle. Der Datensatz kann im Betrieb nachgeladen werden mit allen Werten.
- Die Eingänge zählen jeweils von 0 bis zum eingestellten Sollwert und dann nahtlos wieder bei 0.
- Die Ausgänge werden mit einer Flanke in einem Bit pro Kanal in der Steuerschnittstelle gestartet und durch das Erreichen des Sollwerts wieder zurückgenommen. Es ist mit einem gemeinsamen Parameter für alle Kanäle einstellbar, ob ein Ausgang mit high aktiv ist oder mit low. Wird vor Erreichen des Sollwerts das Bit in der Steuerschnittstelle (SS) zurückgenommen wird auch der Ausgang wieder deaktiviert

- In der Rückmeldeschnittstelle wird der Zustand jedes Ausgangs angezeigt. Zusätzlich für jeden Kanal der aktuelle Zählwert oder die aktuelle Durchflussgeschwindigkeit angezeigt. Der Wert wird über ein Steuerbit in der Steuerschnittstelle ausgewählt. Der Wert für die Durchflussgeschwindigkeit könnte auf Ink/10s hochgerechnet werden, um eine entsprechende Auflösung zu erreichen. Ein bit in der SS zeigt an, welcher Wert gemeldet wird.

10 TODO

->A/B einbauen + testfall + andere testfälle (Nur A) ->Kanalbelegungen nachfragen (erstmal nur DI/DIQ) ->Grenzwertüberschreitung von enc_counter, wenn upperlimit < enc_counter mit in betracht ziehen. Reset wenn wert änderung bei upper_limit? Muss nicht, ist gewolltes Verhalten, auch nicht prüfen

->Reset über CTRL_IF (Sollte es Upper_Limit zurücksetzen?) ->UserWriteREC für die Obergrenzen einbauen (davor über CTRL_IF0(31 downto 16)/ Neue Anforderungen: Obergrenze muss nicht im Betrieb geändert werden) -> von 2 Byte auf 4 Byte durch mehr Speicher möglich ->Counterreset und Enable auf bits zuweisen (ähnlich wie reset) ->Test für overflowdetection ist useless

->UserWriteREC simulieren und Testen und mit fsm stabiler machen ->load,reset,hold logik überarbeiten(wann wird was ausgeführt und wodurch -> Muss nach einem Reset ein Load kommen um upperlimit neu zu laden (schaue beim obrgiegen simulieren)?)

Feedback gespräch -Diagramm für Ablauf -was hab ich mir gedacht -rst einbauen -Anforderungen mit aufschreiben

-Fragen dos donts ->Fault and diagnostic evaluation? -> DQ Force = 0 oder auch noch FB_IF? ->asynchronous signal WR_REC_NEW to the CLK domain ->CTRL_IF Signale auf variable zuweisen oder direkt verwenden? alias verwenden

Antwort: ->CTRL_IF für fehler quittieren, wenn overflow zu schnell toggelt ->ganzes paket ins netzwerklaufwerk des teams und diagramm und link des Product backlog ->generate für weniger logik (weniger kanäle/flexibel)

tobi berichten, dass durch test es sich etwas nach hinten verschiebt

->Review des Codes ->Quartus Compilieren und zum test TGEN (Am Ende)-> Test und angeschlossenden Enc_Enum (Gerät Geber-Simulation) (Davor evtl mit sich selbst DQ auf DI) ->Nightly Test einbauen

->wird wr_rec00 richtig gesetzt? testen

-FB_IF Finalform + Steuerbit -Load new and better Version

->RS422 für inc_enc einbauen

11 Tests

rückgabe über das (fb_if) feedback_interface

EncEmu encoder für impulse

schwierigkeit: di1 oder di0 zählen

RD_Rec sind azyklisch und nur selten benutzt

library test 8 counter -> name mit test cases Jürgen, ..., Kollege aus Prag

Muss ich nicht allein schreiben

Mathias fragen wenn funktionierendes Programm -> in den Nightly test muss in den nightly build

-systemlogic updaten auf v2 / DONE -zähler des sea teams verwenden / -Questa Simulation / (Questa DUT=Design under Test)

- 0000_0000_0000_0000_0000_0000_0000_0001 -> FB_IF(0)(0)<='1'; - /-> Q0.0 (FB_IF(0))

- FB_IF(1)(0) == QD4

-mathias jupiter.tfs.siemens.net für nightly build (muss ins git)

-adress register verbessern

-quellcode verwaltung?

Reaktion auf falsche Parameter Verifikation der korrekten Initialisierung des Encoders nach LOAD Grundkonfiguration und Initialisierung -> System-Reset wird ausgelöst

Verschiedene Zählmodi oder Edge-Typen?

- Zählen mit A oder A&B oder A&B&N -> nicht nötig

- Betriebsart: periodisches Zählen, einmaliges Zählen, usw. -> nicht nötig
- hoch/runterzählen, Invertieren der Hauptzählrichtung -> nicht nötig
- Test des Verhaltens beim Überlauf an Zählgrenzen (oben/unten) -> abhängig von Betriebsart -> Überlauf ist getestet/ Betriebsart ist unwichtig, da nur Hochgezählt wird (B=Low)
- Start nicht mit Wert 0 sondern Startwert -> Machbar aber muss aber nicht
- invertieren der A/B/N Eingangssignale -> ?
- Test der verschiedenen Resetquellen -> Muss

undefinds werden kopiert -> am start muss next_upper_limit initialisiert werden mit einer 0, sonst wird bei jedem load bevor WR_REC_NEW gemacht wird, next_upper_limi auf undefiend gesetzt.

12 Passwörter

PLC: Siemens123

- [Masterarbeit von Andreas Kick](#)
- [Bachelorarbeit von Thomas Weisel](#)
- [Prof. Meitner Zoom Link](#)
- [Enc_Emu](#)

Stimmen die zeitangaben in der bachelorarbeit?